Fix or Remove Sender Algorithm Customization

Document #:	P3826R2 [Latest] [Status]
Date:	2025-11-07
Project:	Programming Language C++
Audience:	SG1 Concurrency and Parallelism Working Group LEWG Library Evolution Working Group LWG Library Working Group
Reply-to:	Eric Niebler <eric.niebler@gmail.com>

Contents

§ 1 Background

In the current Working Draft, 33 [exec] has sender algorithms that are customizable. While the sender/receiver concepts and the algorithms themselves have been stable for several years now, the customization mechanism has seen a fair bit of recent churn. [P3718R0] is the latest effort to shore up the mechanism. Unfortunately, there are gaps in its proposed resolution. This paper details those gaps.

The problem and its solution are easy to describe, but the changes are not trivial. The fix has been implemented once, and another independent implementation is in progress at the time of writing. This paper proposes to fix the issue for C++26. Should the fix be deemed too risky, this paper also offers guidance on how to remove the ability to customize sender algorithms for C++26 in a way that permits us to add it back in C++29.

§ 2 The problem with P3718

[P3718R0] identifies real problems with the status quo of sender algorithm customization. It proposes using information from the sender about where it will *complete* during "early" customization, which happens when a sender algorithm constructs and returns a sender; and it proposes using information from the receiver about where the operation will *start* during "late" customization, when the sender and the receiver are connected.

The problem with this separation of responsibilities is:

Many senders do not know where they will complete until they know where they will be started.

A simple example is the `just()` sender; it completes inline wherever it is started. The information about where a sender will start is not known during early customization, when the sender is being asked for this information.

And even if we knew where the sender will start, there is no generic interface for asking a sender where it will complete *given where it will start*. There currently is no such API, which is the whole problem in a nutshell.

§ 2.1 An illustrative example

This section illustrates the above problem by walking through the algorithm selection process proposed by P3718. Consider the following example:

```
namespace ex = std::execution;
auto sndr = ex::starts_on(gpu, ex::just()) | ex::then(fn);
std::this_thread::sync_wait(std::move(sndr));
```

… where `gpu` is a scheduler that runs work (unsurprisingly) on a GPU.

`fn` will execute on the GPU, so a GPU implementation of `then` should be used. By the proposed resolution of P3718, algorithm customization proceeds as follows:

— During early customization, when `starts_on(gpu, just()) | then(fn)` is executing, the `then` CPO asks the `starts_on(gpu, just())` sender where it will complete as if by:

```
auto& [_, fn, child1] = *this; // *this is a then sender
                               // child1 is a starts_on
        sender
```

```
auto dom1 = ex::get_domain(ex::get_env(child1));
```

— The `starts_on` sender will in turn ask the `just()` sender, as if by:

```
auto& [_, sch, child2] = *this;   // *this is a starts_on
        sender
                                  // child2 is a just sender
auto dom2 = ex::get_domain(ex::get_env(child2));
```

As discussed, the `just()` sender doesn't know where it will complete until it knows where it will be started, but that information is not yet available. As a result, `dom2` ends up as `default_domain`, which is then reported as the domain for the `starts_on` sender. That's incorrect. The `starts_on` sender will complete on the GPU.

— The `then` CPO uses `default_domain` to find an implementation of the `then` algorithm, which will find the default implementation. As a result, the `then` CPO returns an ordinary `then` sender.

— When that `then` sender is connected to `sync_wait`'s receiver, late customization happens. `connect` asks `sync_wait`'s receiver where the `then` sender will be started. It does that with the query `get_domain(get_env(rcvr))`. `sync_wait` starts operations on the current thread, so the `get_domain` query will return `default_domain`. As with early customization, late customization will also not find a GPU implementation.

The end result of all of this is that a default (which is effectively a CPU) implementation will be used to evaluate the `then` algorithm on the GPU. That is a bad state of affairs.

§ 3 Solutions considered

Here is a list of possible ways to address this problem for C++26, sorted in descending awfulness.

§ 3.1 Remove all of the C++26 `std::execution` additions

Although the safest option, I hope most agree that such a drastic step is not warranted by this issue. Pulling the `sender` abstraction and everything that depends on it would result in the removal of:

— The sender/receiver-related concepts and customization points, without which the ecosystem will have no shared async abstraction, and which will set back the adoption of structured concurrency three years.

— The sender algorithms, which capture common async patterns and make them reusable,

— `execution::counting_scope` and `execution::simple_counting_scope`, and related features for incremental adoption of structured concurrency,

— `execution::parallel_scheduler` and all of its related APIs, and

— `execution::task` and `execution::task_scheduler` (C++26 will *still* not have a standard coroutine task type <*heavy sigh*>).

This option should only be considered if all the other options are determined to have unacceptable risk.

§ 3.2 Remove all of the customizable sender algorithms

This option would keep all of the above library components with the exception of the customizable sender algorithms:

— `then`, `upon_error`, `upon_stopped`

— `let_value`, `let_error`, `let_stopped`

— `bulk`, `bulk_chunked`, `bulk_unchunked`

— `starts_on`, `continues_on`, `on`

— `when_all`, `when_all_with_variant`

— `stopped_as_optional`, `stopped_as_error`

— `into_variant`

— `sync_wait`

— `affine_on`

This would leave users with no easy standard way to start work on a given execution context, or transition to another execution context, or to execute work in parallel, or to wait for work to finish.

In fact, without the `bulk` algorithms, we leave no way for the `parallel_scheduler` to execute work in parallel!

While still delivering a standard async abstraction with minimal risk, the loss of the algorithms would make it *just* an abstraction. Like coroutines, adoption of senders as an async *lingua franca* will be hampered by lack of standard library support.

§ 3.3 Remove sender algorithm customization

In this option, we ship everything currently in the Working Draft but remove the ability to customize the algorithms. This gives us a free hand to design a better customization mechanism for C++29 – provided we have confidence that those new customization hooks can be added without break existing behavior.

This option is not as low-risk as it may seem. Firstly, it is difficult to be confident that algorithm customization can be added back without breaking code. Improved customization hooks have been implemented, and wording for the removal has been written, to the best

of the author's ability, such that that the new hooks can be standardized without breaking changes.

Secondly, algorithm customizability is a load-bearing feature. Taking it out is not hard but it isn't trivial either. Customizability is used by the `parallel_scheduler` to accelerate the `bulk` family of algorithms. Although the `task_scheduler` does not currently customize `bulk`, it should. Some design work is necessary before algorithm customization can be removed.

The section Removing algorithm customization describes the effects and possible remedies of the removal option.

§ 3.4 Ship everything as-is and fix algorithm customization in a DR

This option is not as reckless as it sounds. We have a fix and the fix has been implemented in a working and publicly available `execution` library (CCCL). It would not be the first time the Committee shipped a standard with known defects, and the DR process exists for just this purpose.

One potential problem is that, as DRs go, this one would be large-ish. I do not know if this presents a problem procedurally. If it does, then fixing the problem now would make more sense. Any future DRs are likely to be smaller.

§ 3.5 Fix algorithm customization now

This is the option this paper proposes. The fix is easy to describe:

When asking a sender where it will complete, *tell it where it will start*.

That is done by passing the receiver's environment when asking the sender for its completion domain. Instead of `get_domain(get_env(sndr))`, the query would be `get_domain(get_env(sndr), get_env(rcvr))` (but with a query other than `get_domain`, read on).

That change has some ripple effects, the biggest of which is that the receiver is not known during early customization. Therefore, early customization is irreparably broken and must be removed.

> There are no algorithms in `std::execution` that are affected by the removal of early customization since they all do their work lazily. Should a future algorithm be added that eagerly connects a sender, that algorithm should accept an optional execution environment by which users can provide the starting domain. That is not onerous.

There are other ripples from the proposed change. They are described in full detail in section Fixing algorithm customization.

There are risks with trying to fix the problem now. It is a design change happening uncomfortably close to the release of C++26. One mitigating factor is that it is unlikely that the fix would make things more broken than they already are. If there are lingering problems, they could be fixed with the usual DR process.

This fix has been implemented in NVIDIA's CCCL library. At the time of writing a second implementation is being developed independently for stdexec, the `std::execution` reference implementation with an eye to having a few months deployment experience prior to the upcoming London meeting.

§ 4 Fixing algorithm customization

Selecting the right implementation of an algorithm requires requires two things:

1. Identifying the starting and completing domain of the algorithm's async operation, and

2. Using that information to select the preferred implementation for the algorithm that operation represents.

Let's take these two separately.

§ 4.1 Determining the starting and completing domains

As described in Fix algorithm customization now, so-called "early" customization, which determines the return type of `then(sndr, fn)` for example, is irreparably broken. It needs the sender to know where it will complete, which it can't in general.

So the first step is to remove early customization. There is no plan to add it back later.

That leaves "late" customization, which is performed by the `connect` customization point. The receiver, which is an extension of caller, knows where the operation will start. If the sender is given this information – that is, if the sender is told where it will start – it can accurately report where it will complete. This is the key insight.

When `connect` queries a sender's attributes for its completion domain, it should pass the receiver's environment. That way a sender has all available information when computing its completion domain.

§ 4.1.1 `get_completion_domain`

It is sometimes the case that a sender's value and error completions can happen on different domains. For example, imagine trying to schedule work on a GPU. If it succeeds, you

are in the GPU domain, Bob's your uncle. If scheduling fails, however, the error cannot be reported on the GPU because we failed to make it there!

So asking a sender for a singular completion domain is not flexible enough.

When asking for a completion *scheduler*, we have three queries, one for each completion disposition: `get_completion_scheduler<set_[value|error|stopped]_t>`. Similarly, we should have three separate queries for a sender's completion domain: `get_completion_domain<set_[value|error|stopped]_t>`.

> **ASIDE:**
> If we have the `get_completion_scheduler` queries, why do we need `get_completion_domain`? We can ask the completion scheduler for its domain, right? The answer is that there are times when a sender's completion domain is knowable but the completion scheduler is not. E.g., `when_all(s1, s2)` completes on the completion scheduler of either `s1` or `s2`, so its completion scheduler is indeterminate. But if `s1` and `s2` have the same completion *domain*, then we know that `when_all` will complete in that domain.

The addition of the completion domain queries creates a nice symmetry as shown in the table below (with additions in green):

	Receiver	**Sender**

Query for scheduler	`get_scheduler`	`get_completion_scheduler<set_value_t>`
		`get_completion_scheduler<set_error_t>`
		`get_completion_scheduler<set_stopped_t>`

Query for domain	`get_domain`	`get_completion_domain<set_value_t>`
		`get_completion_domain<set_error_t>`
		`get_completion_domain<set_stopped_t>`

For a sender `sndr` and an environment `env`, we can get the sender's completion domain as follows:

```
auto completion_domain = get_completion_domain<set_value_t>
  (get_env(sndr), env);
```

A sender like `just()` would implement this query as follows:

```
struct just_attrs
{
  auto query(get_completion_domain_t<set_value_t>, const auto&
      env) const noexcept
  {
    // an inline sender completes where it starts. the domain of
        the environment is where
    // the sender will start, so return that.
    return get_domain(env);
  }
  //...
```

```
};

template<class... Values>
struct just_sender
{
  //...
  auto get_env() const noexcept
  {
    return just_attrs{};
  }
  //...
};
```

> **Note:**
> A query that accepts an additional argument is novel in `std::execution`, but the query system was designed to support this usage. See 33.2.2 [exec.queryable.concept].

Just as the `get_completion_domain` queries accept an optional `env` argument, so too should the `get_completion_scheduler` queries.

§ 4.2 Algorithm dispatching in `connect`

With the addition of the `get_completion_domain<*>` queries that can accept the receiver's environment, `connect` can now know the starting and completing domains of the async operation it is constructing. When passed arguments `sndr` and `rcvr`, the starting domain is:

```
// Get the operation's starting domain:
auto starting_domain = get_domain(get_env(rcvr));
```

To get the completion domain (when the operation completes successfully):

```
// Get the operation's completion domain for the value channel:
auto   completion_domain   =   get_completion_domain<set_value_t>
        (get_env(sndr), get_env(rcvr));
```

Now `connect` has all the information it needs to select the correct algorithm implementation. Great!

But this presents the `connect` function with a dilemma: how does it use *two* domains to pick *one* algorithm implementation?

Consider that the starting domain might want a say in how `start` works, and the completing domain might want a say in how `set_value` works. So should we let the starting domain customize `start` and the completing domain customize `set_value`?

No. `start` and `set_value` are bookends around an async operation; they must match. Often `set_value` needs state that is set up in `start`. Customizing the two independently is madness.

§ 4.2.1 Solving the double-dispatch problem

The solution is to use sender transforms. Each domain can apply its transform in turn. I do not have reason to believe the order matters, but it is important that when asked to transform a sender, a domain knows whether it is the "starting" domain or the "completing" domain.

Here is how a domain might customize `bulk` when it is the completing domain:

```
struct thread_pool_domain
{
  template<sender-for<bulk_t> Sndr, class Env>
   auto transform_sender(set_value_t, Sndr&& sndr, const Env&
        env) const
  {
    //...
  }
};
```

Since it has `set_value_t` as its first argument, this transform is only applied when `thread_pool_domain` is an operation's completion domain. Had the first argument been `start_t`, the transform would only be used when `thread_pool_domain` is a starting domain.

§ 4.2.2 `transform_sender`

In this proposed design, the `connect` CPO does a few things:

1. Determines the starting and completing domains,
2. Applies the completing domain's transform (if any),
3. Applies the starting domain's transform (if any) to the resulting sender,
4. Connnects the twice-transformed sender to the receiver.

The first three steps are doing something different than connecting a sender and receiver, so it makes sense to factor them out into their own utility. As it so happens we already have such a utility: `transform_sender`.

The proposal requires some changes to how `transform_sender` operates. This new `transform_sender` still accepts a sender and an environment, but it no longer accepts a domain. It computes the two domains and applies the two transforms, recursing if a transform changes the type of the sender.

A possible implementation of `transform_sender` is listed in Appendix A: Listing for updated transform_sender.

With the definition of `transform_sender` in Appendix A, `connect(sndr, rcvr)` is equivalent to `transform_sender(sndr, get_env(rcvr)).connect(rcvr)` (except `rcvr` is

evaluated only once).

§ 4.3 Revisiting the problematic example

Let's see how this new approach addresses the problems noted in the motivating example above. The troublesome code is:

```
namespace ex = std::execution;
auto sndr = ex::starts_on(gpu, ex::just()) | ex::then(fn);
std::this_thread::sync_wait(std::move(sndr));
```

An illustrative example describes how the current design and the "fixed" one proposed in [P3718R0] go off the rails while determining the domain in which the function `fn` will execute, causing it to use a CPU implementation instead of a GPU one.

In the new design, when the `then` sender is being connected to `sync_wait`'s receiver, the starting domain will still be the `default_domain`, but when asking the sender where it will complete, the answer will be different. Let's see how:

— When asked for its completion domain, the `then` sender will ask the `starts_on` sender where it will complete, as if by:

```
auto& [_, fn, child1] = *this; // *this is a then sender
                               // child1 is a starts_on
        sender
auto   dom1   =   ex::get_completion_domain<ex::set_value_t>
        (ex::get_env(child1), ex::get_env(rcvr));
```

— In turn, the `starts_on` sender asks the `just` sender where it will complete, *telling it where it will start*. (This is the new bit.) It looks like:

```
auto& [_, sch, child2] = *this;  // *this is a starts_on
        sender
                                 // child2 is a just sender
// ask for the scheduler's completion domain:
auto  sch_dom  =  ex::get_completion_domain<ex::set_value_t>
        (sch, get_env(rcvr));
// construct an env that reflects the fact that child2 will
        be started on sch:
auto env2 = ex::env{ex::prop{ex::get_scheduler, sch},
                    ex::prop{ex::get_domain, sch_dom},
                    ex::get_env(rcvr)};
// pass the new env when asking child2 for its completion
        domain:
auto   dom2   =   ex::get_completion_domain<ex::set_value_t>
        (ex::get_env(child2), env2);
```

— The `just` sender, when asked where it will complete, will respond with the domain on which it is started. In other words, `get_completion_domain<set_value_t>(get_env(just()), env2)` will return `get_domain(env2)`, which is `sch_dom`.

— Having correctly determined that the `then` sender will start on the default domain and complete on the GPU domain, `connect` can select the right implementation for the

`then` algorithm. It does that by calling:

```
return             ex::transform_sender(forward<Sndr>(sndr),
        ex::get_env(rcvr)).connect(forward<Rcvr>(rcvr));
```

The `transform_sender` call will execute the following (simplified):

```
ex::default_domain().transform_sender(
  ex::start,
       gpu_domain().transform_sender(ex::set_value,    sndr,
        ex::get_env(rcvr)),
  ex::get_env(rcvr))
```

where `gpu_domain` is the domain of the `gpu` scheduler. The `default_domain` does not apply any transformation to `then` senders, so this expression reduces to:

```
gpu_domain().transform_sender(ex::set_value,           sndr,
        ex::get_env(rcvr))
```

So, in the new customization scheme, the GPU domain gets a crack at transforming the `then` sender before it is connected to a receiver, as it should.

§ 4.4 Customizing `continues_on` and `schedule_from`

One of the uglier parts of the current algorithm customization design is that it needs special case handling for the `continues_on` and `schedule_from` algorithms. The proposed design gives us an opportunity to clean this up significantly.

§ 4.4.1 Background

In order to transition between two execution contexts, each of which may know nothing about the other, it is necessary to do it in two steps: a transition *from* a context to the default domain, and a transition from the default domain *to* another context. A scheduler can customize either or both of these two steps by customizing the `continues_on` and `schedule_from` algorithms.

The customization of `continues_on` is found using the completion domain of `sndr`, making it the sanctioned way to transition *off of* a context. `schedule_from` finds its customization using the domain of `sch`, making it useful for transitioning *onto* a context.

When not customized, connecting the sender `continues_on(sndr, sch)` performs a switcheroo and connects `schedule_from(sch, sndr)` instead. In that way, both the source and destination contexts get a say in how the execution transfer is mediated.

What if a domain wants to customize `continues_on`? Asking `continues_on(sndr, sch)` for its completion domain will yield the domain of `sch`, but `continues_on` wants to use the completion domain of `sndr`. The usual `transform_sender` mechanism does not seem to cut it.

This is handled in the current draft wording by making `continues_on` a special case in `transform_sender`. In [P3718R0], the special casing is replaced with a `get_domain_override` attribute query, by which the `continues_on` sender can force `transform_sender` to use a different domain.

Both of these solutions are hacks.

§ 4.4.2 A simplification

A better way to solve this problem is to divide responsibilities differently between `continues_on` and `schedule_from`. Suppose that only `continues_on` transfers execution, and `schedule_from` does nothing and only exists so it can be customized. The `continues_on` customization point would look like:

```
constexpr pipeable-adaptor continues_on =
  []<class Sndr, class Sch>(this auto self, Sndr&& sndr, Sch sch)
  {
    return make-sender(self, sch, schedule_from(forward<Sndr>(sndr)));
  };
```

The `schedule_from` customization point would look like this:

```
constexpr auto schedule_from =
  []<class Sndr>(this auto self, Sndr&& sndr)
  {
    return make-sender(self, {}, forward<Sndr>(sndr));
  };
```

Semantically, `schedule_from(sndr)` is equivalent to `sndr`. Crucially, that means that `schedule_from(sndr)` has the same completion domain as `sndr`. And that makes `schedule_from` a great way to customize how to transition *off of* an execution context.

On the other hand, `continues_on(sndr, sch)` completes on the domain of `sch`, making it a great way to customize how to transition *onto* an execution context.

By splitting `continues_on` and `schedule_from` in this way, we obviate the need for any special cases or domain overrides. The usual `transform_sender` mechanism is sufficient.

In the CCCL project, I have implemented this design and ported my CUDA stream scheduler to use it. I needed to customize `schedule_from` for the CUDA stream scheduler to mediate the execution transfer from the GPU back to the CPU. Besides the `bulk` algorithms, `schedule_from` is the only algorithm the GPU scheduler needs to customize.

> NOTE Carving the two algorithms this way flips how they are dispatched. `continues_on` now dispatches based on the domain of `sch`, and `schedule_from` on the completion domain of the predecessor sender. The author believes this is a far more sensible arrangement.

§ 4.5 `inline_scheduler` improvements

The suggestion above to extend the `get_completion_scheduler<*>` query presents an intriguing possibility for the `inline_scheduler`: the ability for it to report the scheduler on which its scheduling operations complete!

Consider the sender `schedule(inline_scheduler())`. Ask it where it completes today and it will say, "I complete on the `inline_scheduler`.", which isn't terribly useful. However, if you ask it, "Where will you complete – and by the way you will be started on the `parallel_scheduler`?", now that sender can report that it will complete on the `parallel_scheduler`.

The result is that code that uses the `inline_scheduler` will no longer cause the *actual* scheduler to be hidden.

This realization is the motivation behind the change to strike the `get_completion_scheduler<set_value_t>(get_env(schedule(sch)))` requirement from the `scheduler` concept. We want that expression to be ill-formed for the `inline_scheduler`. Instead, we want the following query to be well-formed:

```
get_completion_scheduler<set_value_t>(get_env(schedule(in-
        line_scheduler())), get_env(rcvr))
```

That expression should be equivalent to `get_scheduler(get_env(rcvr))`, which says that the sender of `inline_scheduler` completes wherever it is started.

§ 4.6 Indeterminate domains

When computing completion domains, it is sometimes the case that an operation can complete on domain `A` or domain `B` for a given disposition (value, error, or stopped). Imagine such a sender with an indeterminate completion domain for `set_value`. How does algorithm customization work in that case?

First, we recognize that very few algorithms will ever be customized; a given domain may only customize a handful. Given `sndr | then(fn)`, there is no difficulty picking the implementation for `then` even if `sndr` can complete successfully on either domain `A` or `B`, *provided neither domain customizes `then`*.

That insight makes it advantageous for a sender to report *all* the domains on which it might complete for a particular completion channel. It can do that with a new domain type: `indeterminate_domain<Domains...>`, which looks like this:

```
template<class... Domains>
struct indeterminate_domain
{
  template<class Tag, class Sndr, class Env>
  static constexpr auto transform_sender(Tag, Sndr&& sndr, con-
        st Env& env)
  {
    // Mandates: for all D in Domains, the expression
     // D().transform_sender(Tag(), forward<Sndr>(sndr), env) is
        either ill-formed or else
```

```
    // has the same type as
      // default_domain().transform_sender(Tag(), forward<Sndr>
        (sndr), env)
         return default_domain().transform_sender(Tag(), for-
        ward<Sndr>(sndr), env);
  }
};
```

Given an environment `e`, a sender like `when_all(sndrs...)` would have a value completion domain of

```
COMMON-DOMAIN(COMPL-DOMAIN(set_value_t, sndrs, e)...)
```

where:

— *COMPL-DOMAIN*`(T, S, E)` is the type of `get_completion_domain<T>(get_env(S), E)` if that expression is well-formed, or `indeterminate_domain<>()` otherwise, and

— *COMMON-DOMAIN*`(Ds...)` is `common_type_t<Ds...>` if that expression is well-formed, and `indeterminate_domain<Ds...>` otherwise.

The final piece is to specialize `common_type` such that `indeterminate_domain<As...>` and `indeterminate_domain<Bs...>` have a common type of `indeterminate_domain<As..., Bs...>`, and such that `common_type_t<indeterminate_domain<As...>, D>` is `indeterminate_domain<As..., D>`.

§ 4.7 The procedure for the fix

The steps for fixing algorithm customization are detailed below.

1. Remove the uses of `transform_sender` in the sender adaptor algorithm customization points (33.9.12 [exec.adapt]). Directly return the result of calling *make-sender* rather than passing it to `transform_sender`.

2. Remove the exposition-only helpers:

   — *completion-domain* (33.9.2 [exec.snd.expos]/8-9),

   — *get-domain-early* (33.9.2 [exec.snd.expos]/13), and

   — *get-domain-late* (33.9.2 [exec.snd.expos]/14).

3. Add the `get_completion_domain` queries:

   — `get_completion_domain<set_value_t>`

   — `get_completion_domain<set_error_t>`

   — `get_completion_domain<set_stopped_t>`

4. Change the `get_completion_scheduler` queries to accept an optional environment argument.

5. Make the `get_domain(env)` query smarter by falling back to the current scheduler's domain if `env.query(get_domain)` is ill-formed, and falling back further to `default_domain()` if `env` does not have a current scheduler.

6. Restore the ability of `env<...>::query` to accept additional arguments.

7. Rename the current `schedule_from` algorithm to `continues_on` and change it to return *`make-sender`*`(continues_on, sch, schedule_from(sndr))`, where `schedule_from` is a new algorithm such that `schedule_from(sndr)` is equivalent to *`make-sender`*`(schedule_from, {}, sndr)`.

8. Remove the (unused) `transform_env` function and the `transform_env` members of the sender algorithm CPOs and from `default_domain`.

9. Change `transform_sender` from `transform_sender(Domain, Sndr, Env...)` to `transform_sender(Sndr, Env)`. Have it compute the sender's starting and completing domains and apply their transforms to `Sndr` as shown in Appendix A: Listing for updated transform_sender.

10. Update the usages of `transform_sender` in `connect` and `get_completion_signatures` to reflect its new signature.

11. For the `transform_sender` member functions in the sender algorithm CPOs, add `set_value_t,` in the front of their parameter list. Parameterize the `transform_sender` member in `default_domain` with a leading `Tag` parameter.

12. Add a class template `indeterminate_domain<Domains...>` as described in Indeterminate domains.

13. Update the attributes of the sender algorithms to properly report their completion schedulers and completion domains given an optional `env` argument. Also update the `inline_scheduler` and its schedule-sender to compute their completion scheduler and domain from the extra `env` argument.

14. From the `scheduler` concept, replace the required expression

```
{ auto(get_completion_scheduler<set_value_t>(get_env(sched-
        ule(std::forward<Sch>(sch))))) }
    -> same_as<remove_cvref_t<Sch>>;
```

    with a semantic requirement that *if* the above expression is well-formed – which it is for the `parallel_scheduler`, the `task_scheduler`, and `run_loop`'s scheduler – then it shall compare equal to `sch`. (See inline_scheduler improvements for the motivation behind these changes.)

15. Simplify the `on` algorithm, which no longer needs to use `write_env` to make child operations aware of the current scheduler and domain.

§ 4.8 Proposed wording, option 1: Fix algorithm customization

[Editor's note: Introduce the notion of “execution domains” by changing 33.3 [exec.async.ops#1] as follows:]

[Editor's note: In 33.4 [execution.syn], make the following changes:]

```
… as before …

namespace std::execution {
  // [exec.queries], queries
  struct get_domain_t { unspecified };
  struct get_scheduler_t { unspecified };
  struct get_delegation_scheduler_t { unspecified };
  struct get_forward_progress_guarantee_t { unspecified };
  template<class CPO>
    struct get_completion_scheduler_t { unspecified };
  template<class CPO>
    struct get_completion_domain_t { unspecified };
  struct get_await_completion_adaptor_t { unspecified };

  inline constexpr get_domain_t get_domain{};
  inline constexpr get_scheduler_t get_scheduler{};
          inline      constexpr      get_delegation_scheduler_t
        get_delegation_scheduler{};
  enum class forward_progress_guarantee;
        inline    constexpr    get_forward_progress_guarantee_t
        get_forward_progress_guarantee{};
  template<class CPO>
                   constexpr     get_completion_scheduler_t<CPO>
        get_completion_scheduler{};
  template<class CPO>
                     constexpr      get_completion_domain_t<CPO>
        get_completion_domain{};
         inline     constexpr     get_await_completion_adaptor_t
        get_await_completion_adaptor{};

  struct get_env_t { unspecified };
  inline constexpr get_env_t get_env{};

  template<class T>
    using env_of_t = decltype(get_env(declval<T>()));

  // [exec.prop], class template prop
  template<class QueryTag, class ValueType>
    struct prop;

  // [exec.env], class template env
  template<queryable... Envs>
    struct env;

  // [exec.domain.indeterminate], execution domains
  template<class... Domains>
    struct indeterminate_domain;
```

```
  // [exec.domain.default], execution domains
  struct default_domain;

… as before …

  template<sender Sndr>
    using tag_of_t = see below;

  // [exec.snd.transform], sender transformations
  template<~~class Domain,~~ sender Sndr, queryable~~…~~ Env>
      requires (sizeof...(Env) <= 1)
    constexpr sender decltype(auto) transform_sender(
       ~~Domain dom,~~ Sndr&& sndr, const Env&~~...~~ env) noexcept(see
         below);

  ~~// [exec.snd.transform.env], environment transformations~~
  ~~template<class Domain, sender Sndr, queryable Env>~~
    ~~constexpr queryable decltype(auto) transform_env(~~
      ~~Domain dom, Sndr&& sndr, Env&& env) noexcept;~~

  // [exec.snd.apply], sender algorithm application
  template<class Domain, class Tag, sender Sndr, class... Args>
    constexpr decltype(auto) apply_sender(
      Domain dom, Tag, Sndr&& sndr, Args&&... args) noexcept(see
        below);

  // [exec.connect], the connect sender algorithm
  struct connect_t;
  inline constexpr connect_t connect{};

… as before …
```

[Editor's note: Before subsection 33.5.1 [exec.fwd.env], insert a new subsection with stable name [exec.queries.expos] as follows:]

§ 4.8.1 Query utilities [exec.queries.expos]

1. [exec.queries] makes use of the following exposition-only entities.

2. For subexpressions `q` and `tag` and pack `args`, let *`TRY-QUERY`*`(q, tag, args...)` be expression-equivalent to *`AS-CONST`*`(q).query(tag, args...)` if that expression is well-formed, and *`AS-CONST`*`(q).query(tag)` otherwise.

3. For subexpressions `q` and `tag` and pack `args`, let *`HIDE-SCHED`*`(q)` be an object `o` such that `o.query(tag, args...)` is ill-formed when the decayed type of `tag` is `get_scheduler_t` or `get_domain_t`, and `o.query(tag, args...)` otherwise.

[Editor's note: Change [exec.get.domain] as follows:]

§ `execution::get_domain` [exec.get.domain]

1. `get_domain` asks a queryable object for its associated execution domain tag.

2. The name `get_domain` denotes a query object. For a subexpression `env`, `get_domain(env)` is expression-equivalent to *`MANDATE-NOTHROW`*`(D())`, where `D` is the type of the first of the following expressions that is well-formed [Editor's note: Reformatted as a list.]

(2.1) — ~~*`MANDATE-NOTHROW(`*~~`auto(`*`AS-CONST`*`(env).query(get_domain))`

(2.2) — `get_completion_domain<set_value_t>(get_scheduler(env),` *`HIDE-SCHED`*`(env))`

(2.3) — `default_domain()`

3. `forwarding_query(execution::get_domain)` is a core constant expression and has value `true`.

[Editor's note: Change subsection 33.5.6 [exec.get.scheduler] as follows:]

1. `get_scheduler` asks a queryable object for its associated scheduler.

2. The name `get_scheduler` denotes a query object. For a subexpression `env`, `get_scheduler(env)` is expression-equivalent to

```
get_completion_scheduler<set_value_t>(MANDATE-
        NOTHROW(AS-CONST(env).query(get_sched-
        uler)), HIDE-SCHED(env))
```

Mandates: If the expression above is well-formed, its type satisfies `scheduler`.

3. `forwarding_query(execution::get_scheduler)` is a core constant expression and has value `true`.

[Editor's note: Change subsection 33.5.9 [exec.get.compl.sched] as follows:]

§ `execution::get_completion_scheduler` [exec.get.compl.sched]

1. `get_completion_scheduler<`*`completion-tag`*`>` obtains the completion scheduler associated with a completion tag from a sender's attributes.

2. For subexpression `sch1` and pack `envs`, let `sch2` be *`TRY-QUERY`*`(sch, get_completion_scheduler<set_value_t>, envs...)` and let *`QUERY-RECURSE`*`(sch1, envs...)` be expression-equivalent to *`QUERY-RECURSE`*`(sch2, envs...)` if that expression is well-formed and `sch1` and `sch2` have different types or compare unequal; otherwise, `sch`.

3. The name `get_completion_scheduler` denotes a query object template. For a subexpression `q` and pack `envs`, the expression `get_completion_scheduler<`*`completion-tag`*`>(q, envs...)` is ill-formed if *`completion-tag`* is not one of `set_value_t`, `set_error_t`, or `set_stopped_t`. Otherwise, `get_completion_scheduler<`*`completion-tag`*`>(q, envs...)` is expression-equivalent to

```
~~MANDATE-NOTHROW(AS-CONST(q).query(get_completion_scheduler<completion-tag>))~~
```

(3.1) — *`MANDATE-NOTHROW(RECURSE-QUERY(TRY-QUERY(`*`q, get_completion_scheduler<`*`completion-tag`*`>, envs...), envs...))`* if that expression is well-formed.

(3.2) — Otherwise, `auto(q)` if the type of `q` satisfies `scheduler` and `sizeof...(envs) != 0` is `true`.

(3.3) — Otherwise, `get_completion_scheduler<`*`completion-tag`*`>(q, envs...)` is ill-formed.

Mandates: If ~~the expression above~~ `get_completion_scheduler<`*`completion-tag`*`>(q, envs...)` is well-formed, its type satisfies `scheduler`.

4. For a type `Tag`, subexpression `sndr`, and pack `env`, let `CS` be `completion_signatures_of_t<decay_t<decltype((sndr))>, decltype((env))...>`. If both `get_completion_scheduler<Tag>(get_env(sndr), env...)` and `CS` are well-formed and `CS().`*`count-of`*`(Tag()) == 0` is `true`, the program is ill-formed.

5. Let *`completion-fn`* be a completion function ([exec.async.ops]); let *`completion-tag`* be the associated completion tag of *`completion-fn`*; let `args` and `envs` be ~~a~~ packs of subexpressions; and let `sndr` be a subexpression such that `sender<decltype((sndr))>` is `true` and `get_completion_scheduler<`*`completion-tag`*`>(get_env(sndr), envs...)` is well-formed and denotes a scheduler `sch`. If an asynchronous operation created by connecting `sndr` with a receiver `rcvr` causes the evaluation of *`completion-fn`*`(rcvr, args...)`, the behavior is undefined unless the evaluation happens on an execution agent that belongs to `sch`'s associated execution resource.

6. The expression `forwarding_query(get_completion_scheduler<`*`completion-tag`*`>)` is a core constant expression and has value `true`.

[Editor's note: After subsection 33.5.9 [exec.get.compl.sched], add a new subsection with stable name [exec.get.compl.domain] as follows.]

§ `execution::get_completion_domain` [exec.get.compl.domain]

1. `get_completion_domain<`*`completion-tag`*`>` obtains the completion domain associated with a completion tag from a sender's attributes.

2. The name `get_completion_domain` denotes a query object template. For a subexpression `o` and pack `env`, the expression `get_completion_domain<`*`completion-tag`*`>(o, env...)` is ill-formed if *`completion-tag`* is not one of `set_value_t`, `set_error_t`, or `set_stopped_t`. Otherwise, `get_completion_domain<`*`completion-tag`*`>(o, env...)` is expression-equivalent to *`MANDATE-NOTHROW`*`(D())`, where `D` is the type of the first of the following expressions that is well-formed

(2.1) — *`TRY-QUERY`*`(o, get_completion_domain<`*`completion-tag`*`>, env...)`

(2.2) — *`TRY-QUERY`*`(get_completion_scheduler<`*`completion-tag`*`>(o, env...), get_completion_domain<set_value_t>, env...)`

   If none of the above expressions is well-formed, the expression `get_completion_domain<`*`completion-tag`*`>(o, env...)` is ill-formed.

3. For a type `Tag`, subexpression `sndr`, and pack `env`, let `CS` be `completion_signatures_of_t<decay_t<decltype((sndr))>, decltype((env))...>`. If both `get_completion_domain<Tag>(get_env(sndr), env...)` and `CS` are well-formed and `CS().`*`count-of`*`(Tag()) == 0` is `true`, the program is ill-formed.

4. Let *`completion-fn`* be a completion function ([exec.async.ops]); let *`completion-tag`* be the associated completion tag of *`completion-fn`*; let `args` and `env` be packs of subexpressions; and let `sndr` be a subexpression such that `sender<decltype((sndr))>` is `true` and `get_completion_domain<`*`completion-tag`*`>(get_env(sndr), env...)` is well-formed and denotes a domain `d`. If an asynchronous operation created by connecting `sndr` with a receiver `rcvr` causes the evaluation of *`completion-fn`*`(rcvr, args...)`, the behavior is undefined unless the evaluation happens on an execution agent of an execution resource whose associated execution domain tag is `d`.

5. The expression `forwarding_query(get_completion_domain<`*`completion-tag`*`>)` is a core constant expression and has value `true`.

[Editor's note: In 33.6 [exec.sched], change paragraphs 1, 5, and 6 as follows:]

1. The `scheduler` concept defines the requirements of a scheduler type (33.3 [exec.async.ops]). `schedule` is a customization point object that accepts a scheduler. A valid invocation of `schedule` is a schedule-expression.

```
namespace std::execution {
  template<class Sch>
    concept scheduler =
              derived_from<typename  remove_cvre-
        f_t<Sch>::scheduler_concept,      sched-
        uler_t> &&
      queryable<Sch> &&
      requires(Sch&& sch) {
         { schedule(std::forward<Sch>(sch)) } ->
        sender;
        { auto(get_completion_scheduler<set_val-
        ue_t>(
              get_env(schedule(std::forward<Sch>
        (sch)))) }
             -> same_as<remove_cvref_t<Sch>>;
      } &&
         equality_comparable<remove_cvref_t<Sch>>
        &&
      copyable<remove_cvref_t<Sch>>;
}
```

… as before …

5. For a given scheduler expression `sch`, if the expression `get_completion_scheduler<set_value_t>(get_env(schedule(sch)))` is well-formed, it shall compare equal to `sch`.

6. For a given scheduler expression `sch` and pack of subexpressions `env`, if the expression `get_completion_domain<`*`completion-tag`*`>(sch, env...)` is well-formed, then the expression `get_completion_domain<`*`completion-tag`*`>(get_env(schedule(sch)), env...)` is also well-formed and has the same type.

5. For a given scheduler expression `sch` and pack of subexpressions `env`, if the expression `get_completion_scheduler<`*`completion-tag`*`>(sch, env...)` is well-formed, then the expression `get_completion_scheduler<`*`completion-tag`*`>(get_env(schedule(sch)), env...)` is also well-formed and has the same type and value.

[Editor's note: Change 33.9.2 [exec.snd.expos] paragraph 3 as follows:]

3. For a query object `q` ~~and~~, a subexpression `v`, and a pack of subexpressions `as`, *`MAKE-ENV`*`(q, v)` is an expression `env` whose type satisfies *`queryable`* such that the result of `env.query(q, as...)` has a value equal to `v` (18.2 [concepts.equality]). Unless otherwise stated, the object to which `env.query(q, as...)` refers remains valid while `env` remains valid.

[Editor's note: Before 33.9.2 [exec.snd.expos] paragraph 6, add two new paragraph as follows:]

6. For a pack of subexpressions `domains`, *`COMMON-DOMAIN`*`(domains...)` is expression-equivalent to `common_type_t<decltype(auto(domains))...>()` if that expression is well-formed, and `indeterminate_domain<Ds...>()` otherwise, where `Ds` is the pack of types consisting of `decltype(auto(domains))...` with duplicate types removed.

7. For type `Tag`, subexpression, `sndr` and pack `env`, *`COMPL-DOMAIN`*`(Tag, sndr, env)` is expression-equivalent to `D()` where `D` is the type of `get_completion_domain<Tag>(get_env(sndr), env...)` if that expression is well-formed or if `sizeof...(env) == 0` is `true`, and `indeterminate_domain()` otherwise.

[Editor's note: Replace 33.9.2 [exec.snd.expos] paragraph 6 about *`SCHED-ATTRS`* and *`SCHED-ENV`* with the following paragraphs:]

8. For unqualified type `Tag` and subexpressions `sch`, `sndr`, and pack `as`, *`SCHED-ATTRS`*`(sch, sndr)` is an expression `o` whose type satisfies *`queryable`* such that `o.query(Tag(), as...)` is expression-equivalent to:

(8.1) — `Tag()(sch, as...)` if `Tag` is a specialization of `get_completion_scheduler_t` or `get_completion_domain_t` and either of the following is `true`:

(8.1.1) — The template parameter of `Tag` is `set_value_t`, or

(8.1.2) — *`never-completes-with`*`<Sndr, `*`completion-tag`*`, decltype(as)...>` is `true`, where *`completion-tag`* is the template parameter of `Tag`, and `Sndr` is `decay_t<decltype((sndr))>`.

(8.2) — `D()` if `Tag` is `get_completion_domain_t<`*`completion-tag`*`>`, where `D` is the type of the expression

```
COMMON-DOMAIN(COMPL-DOMAIN(completion-tag,
        sndr, as...),
                COMPL-DOMAIN(completion-tag,
        schedule(sch), FWD-ENV(as)...))
```

(8.3) — Otherwise, *`FWD-ENV`*`(get_env(sndr)).query(Tag(), as...)` if either of the following is `true`:

(8.3.1) — `Tag` is not a specialization of `get_completion_scheduler_t` or `get_completion_domain_t`, or

(8.3.2) — *`never-completes-with`*`<schedule_result_t<decltype((sch))>, `*`comple-`*

tion-tag, `decltype(as)...>` is `true`, where *completion-tag* is the template parameter of `Tag`.

(8.4) — Otherwise, the expression `o.query(Tag(), as...)` is ill-formed.

9. For a scheduler `sch`, *SCHED-ENV*`(sch)` is an expression `o` whose type satisfies *queryable* such that

(9.1) — `o.query(get_scheduler)` is a prvalue with the same type and value as `sch`,

(9.2) — `o.query(get_domain)` is expression-equivalent to `sch.query(get_domain)` if that expression is well-formed, and `default_domain()` otherwise.

[Editor's note: Remove the prototype of the exposition-only *completion-domain* function just before 33.9.2 [exec.snd.expos] paragraph 8, and with it remove paragraphs 8 and 9, which specify the function's behavior.]

[Editor's note: Remove 33.9.2 [exec.snd.expos] paragraphs 13 and 14 and the prototypes for the *get-domain-early* and *get-domain-late* functions.]

[Editor's note: After 33.9.2 [exec.snd.expos] paragraph 47 (*not-a-sender*), add the following new paragraph]

48.
```
struct not-a-scheduler {
  using scheduler_concept = scheduler_t;

  constexpr auto schedule() const noexcept {
    return not-a-sender();
  }
};
```

[Editor's note: Add the following new paragraphs after 33.9.2 [exec.snd.expos] paragraph 50 as follows:]

51.
```
template<class Sndr, class SetTag, class... Env>
  concept never-completes-with = requires {
    requires (0 == get_completion_signatures<Sndr, Env...>
        ().count-of(SetTag()));
  };
```

```
template<class Fn, class Default, class... Args>
   constexpr auto call-with-default(Fn&& fn, Default&& value,
        Args&&... args) noexcept(see below);
```

52. Let `e` be the expression `std::forward<Fn>(fn)(std::forward<Args>(args)...)` if that expression is well-formed; otherwise, it is `static_cast<Default>(std::forward<Default>(value))`.

53. *Returns*: `e`.

54. *Remarks*: The expression in the `noexcept` clause is `noexcept(e)`.

55.
```
template<class Tag>
  struct inline-attrs {
    see below
  };
```

56. For a subexpression `env`, `inline-attrs<Tag>{}.query(get_completion_scheduler<Tag>, env)` is expression-equivalent to `get_scheduler(env)`.

57. For a subexpression `env`, `inline-attrs<Tag>{}.query(get_completion_domain<Tag>, env)` is expression-equivalent to `get_domain(env)`.

[Editor's note: Add a new subsection before [exec.domain.default] with stable name [exec.domain.indeterminate] as follows:]

§ 33.9.? `execution::indeterminate_domain` [exec.domain.indeterminate]

1.

```
namespace std::execution {
  template<class... Domains>
    struct indeterminate_domain {
      indeterminate_domain() = default;
      constexpr indeterminate_domain(auto&&) noexcept {}

      template<class Tag, sender Sndr, queryable Env>
                    static constexpr sender decltype(auto)
        transform_sender(Tag, Sndr&& sndr, const Env& env)
          noexcept(see below);
    };
}
```

```
template<class Tag, sender Sndr, queryable Env>
   static constexpr sender decltype(auto) transform_sender(Tag,
        Sndr&& sndr, const Env& env)
    noexcept(see below);
```

2. *Mandates*: For each type `D` in `Domains...`, the expression `D().transform_sender(Tag(), std::forward<Sndr>(sndr), env)` is either ill-formed or has the same decayed type as `default_domain().transform_sender(Tag(), std::forward<Sndr>(sndr), env)`.

3. *Returns*: `default_domain().transform_sender(Tag(), std::forward<Sndr>(sndr), env)`

4. *Remarks*: For a pack of types `Ds`, `common_type_t<indeterminate_domain<Domains...>, indeterminate_domain<Ds...>>` is indetermi-

nate_domain<Us...> where Us is the pack of types in Domains..., Ds... except with duplicate types removed. For a type D that is not a specialization of indeterminate_domain, common_type_t<indeterminate_domain<Domains...>, D> is D if sizeof...(Domains) == 0 is true, and common_type_t<indeterminate_domain<Domains...>, indeterminate_domain<D>> otherwise.

[Editor's note: Change 33.9.5 [exec.domain.default] as follows:]

§ 33.9.5 `execution::default_domain` [exec.domain.default]

1.

```
namespace std::execution {
  struct default_domain {
    template<class Tag, sender Sndr, queryable... Env>
        requires (sizeof...(Env) <= 1)
          static constexpr sender decltype(auto) transfor-
        m_sender(Tag, Sndr&& sndr, const Env&... env)
        noexcept(see below);

    template<sender Sndr, queryable Env>
                  static constexpr queryable decltype(auto)
        transform_env(Sndr&& sndr, Env&& env) noexcept;

    template<class Tag, sender Sndr, class... Args>
       static constexpr decltype(auto) apply_sender(Tag, Sndr&&
        sndr, Args&&... args)
        noexcept(see below);
  };
}
```

```
template<class Tag, sender Sndr, queryable... Env>
    requires (sizeof...(Env) <= 1)
   static constexpr sender decltype(auto) transform_sender(Tag,
        Sndr&& sndr, const Env&... env)
    noexcept(see below);
```

2. Let e be the expression

```
tag_of_t<Sndr>().transfor-
        m_sender(Tag(),std::forward<Sndr>(sndr),
        env...)
```

if that expression is well-formed; otherwise, static_cast<Sndr>(std::forward<Sndr>(sndr)). [Editor's note: See https://cplusplus.github.io/LWG/issue4368]

3. *Returns*: e.

4. *Remarks*: The exception specification is equivalent to noexcept(e).

~~template<sender Sndr, *queryable* Env>~~
~~constexpr *queryable* decltype(auto) transform_env(Sndr&& sndr, Env&& env) noexcept;~~

5. ~~Let e be the expression~~

~~tag_of_t<Sndr>~~
~~().transform_env(std::forward<Sndr>~~
~~(sndr), std::forward<Env>(env))~~

~~if that expression is well-formed; otherwise, *FWD-ENV*(std::forward<Env>(env)).~~

6. ~~*Mandates*: noexcept(e) is true.~~

7. ~~*Returns*: e.~~

```
template<class Tag, sender Sndr, class... Args>
constexpr   decltype(auto)   apply_sender(Tag,   Sndr&&   sndr,
        Args&&... args)
  noexcept(see below);
```

8. Let e be the expression

```
Tag().apply_sender(std::forward<Sndr>(sndr),
        std::forward<Args>(args)...)
```

9. *Constraints*: e is a well-formed expression.

10. *Returns*: e.

11. *Remarks*: The exception specification is equivalent to noexcept(e).

[Editor's note: Change 33.9.6 [exec.snd.transform] as follows:]

§ execution::transform_sender [exec.snd.transform]

```
namespace std::execution {
  template<~~class Domain,~~ sender Sndr, queryable~~...~~ Env>
      ~~requires (sizeof...(Env) <= 1)~~
       constexpr  sender  decltype(auto)  transform_sender(~~Domain dom,~~ Sndr&& sndr, const Env&~~...~~ env)
      noexcept(see below);
}
```

1. For a subexpression s, let *domain-for*(start, s) be D() where D is the decayed type of get_domain(env) if that expressions that is well-formed, and default_domain otherwise.

2. Let *domain-for*(set_value, s) be D() where D is the decayed type of get_completion_domain<set_value_t>(get_env(sndr), env) if that is well-formed, and default_domain otherwise.

3. Let *transformed-sndr*(dom, tag, s) be the expression

```
~~dom.transform_sender(std::forward<Sndr>(sndr),
        env...)~~
dom.transform_sender(tag, s, env)
```

if that expression is well-formed; otherwise,

```
~~default_domain().transform_sender(std::forward<Sndr>
        (sndr), env...)~~
default_domain().transform_sender(tag, s, env)
```

Let ~~*final-sndr*~~ *transform-recurse*(dom, tag, s) be the expression *transformed-sndr*(dom, tag, s) if *transformed-sndr*(dom, tag, s) and ~~sndr~~ s have the same type ignoring cv-qualifiers; otherwise, it is the expression ~~transform_sender(dom, *transformed-sndr*, env...)~~ *transform-recurse*(dom2, tag, s2) where s2 is *transformed-sender*(dom, tag, s) and d2 is *domain-for*(tag, s2).

Let *tmp-sndr* be the expression

```
transform-recurse(domain-for(set_value,    sndr),
        set_value, sndr)
```

and let *final-sndr* be the expression

```
transform-recurse(domain-for(start,    tmp-sndr),
        start, tmp-sndr)
```

4. *Returns*: *final-sndr*.

5. *Remarks*: The exception specification is equivalent to noexcept(*final-sndr*).

[Editor's note: Remove section 33.9.7 [exec.snd.transform.env].]

[Editor's note: Change 33.9.9 [exec.getcomplsigs] paragraph 1 as follows:]

```
template<class Sndr, class... Env>
   consteval auto get_completion_signatures() -> valid-comple-
        tion-signatures auto;
```

1. Let *except* be an rvalue subexpression of an unspecified class type *Except* such that move_constructible<*Except*> && derived_from<*Except*, exception> is true. Let *CHECKED-COMPLSIGS*(*e*) be *e* if *e* is a core constant expression whose type satisfies *valid-completion-signatures*; otherwise, it is the following expression: (*e*, throw *except*, completion_signatures()) Let *get-complsigs*<Sndr, Env...>() be expression-equivalent to remove_reference_t<Sndr>::template get_completion_signatures<Sndr, Env...>(). Let NewSndr be Sndr if sizeof...(Env)

`== 0` is `true`; otherwise, `decltype(s)` where `s` is the following expression:

```
transform_sender(
   ~~get-domain-late(declval<Sndr>(), declval<Env>()...),~~
  declval<Sndr>(),
  declval<Env>()...)
```

[Editor's note: Change 33.9.10 [exec.connect] paragraph 2 as follows:]

2. The name `connect` denotes a customization point object. For subexpressions `sndr` and `rcvr`, let `Sndr` be `decltype((sndr))` and `Rcvr` be `decltype((rcvr))`, let `new_sndr` be the expression

```
transform_sender(~~decltype(get-domain-late(sndr, get_env(rcvr)))~~{}, sndr, get_env(rcvr))
```

and let `DS` and `DR` be `decay_t<decltype((new_sndr))>` and `decay_t<Rcvr>`, respectively.

[Editor's note: Remove 33.9.11.1 [exec.schedule] paragraph 4 as follows:]

4. ~~If the expression~~

```
~~get_completion_scheduler<set_value_t>(get_env(sch.schedule())) == sch~~
```

~~is ill-formed or evaluates to `false`, the behavior of calling `schedule(sch)` is undefined.~~

[Editor's note: Change 33.9.11.2 [exec.just] paragraph 2 as follows:]

2. The names `just`, `just_error`, and `just_stopped` denote customization point objects. Let *`just-cpo`* be one of `just`, `just_error`, or `just_stopped`. For a pack of subexpressions `ts`, let `Ts` be the pack of types `decltype((ts))`. The expression *`just-cpo`*`(ts...)` is ill-formed if

(2.1) — (*`movable-value`*`<Ts> &&...)` is `false`, or

(2.2) — *`just-cpo`* is `just_error` and `sizeof...(ts) == 1` is `false`, or

(2.3) — *`just-cpo`* is `just_stopped` and `sizeof...(ts) == 0` is `false`.

Otherwise, it is expression-equivalent to *`make-sender`*`(`*`just-cpo`*`,` *`product-type`*`{ts...})`.

For `just`, `just_error`, and `just_stopped`, let *`set-cpo`* be `set_value`, `set_error`, and `set_stopped`, respectively. The exposition-only class

template *impls-for* (33.9.2 [exec.snd.expos]) is specialized for *just-cpo* as follows:

```
namespace std::execution {
  template<>
   struct  impls-for<decayed-typeof<just-cpo>>  :
        default-impls {
    static constexpr auto get-attrs =
        [](const auto& data) noexcept -> inline-
        attrs<decayed-typeof<set-cpo>> {
        return {};
      };

    static constexpr auto start =
        [](auto& state, auto& rcvr) noexcept ->
        void {
        auto& [...ts] = state;
                        set-cpo(std::move(rcvr),
        std::move(ts)...);
      };
  };
}
```

[Editor's note: Change 33.9.11.3 [exec.read.env] paragraph 3 as follows:]

3. The exposition-only class template *impls-for* (33.9.2 [exec.snd.expos]) is specialized for `read_env` as follows:

```
namespace std::execution {
  template<>
   struct  impls-for<decayed-typeof<read_env>>  :
        default-impls {
    static constexpr auto get-attrs =
        [](const auto& data) noexcept -> inline-
        attrs<set_value_t> {
        return {};
      };

    static constexpr auto start =
         [](auto query, auto& rcvr) noexcept ->
        void {
                             TRY-SET-VALUE(rcvr,
        query(get_env(rcvr)));
      };
  };

  template<class Sndr, class Env>
    static consteval void check-types();
}
```

[Editor's note: Change 33.9.12.5 [exec.starts.on] paragraphs 3 and 4, and insert a new paragraph 5 as follows:]

3. Otherwise, the expression `starts_on(sch, sndr)` is expression-equivalent to:

~~transform_sender(~~
~~*query-with-default*(get_domain, sch, default_domain()),~~
make-sender(starts_on, sch, sndr)~~)~~

~~except that sch is evaluated only once.~~

4. Let out_sndr and env be subexpressions such that OutSndr is decltype((out_sndr)). If *sender-for*<OutSndr, starts_on_t> is false, then the expression~~s starts_on.transform_env(out_sndr, env) and~~ starts_on.transform_sender(set_value,out_sndr, env) ~~are~~is ill-formed; otherwise

(4.1) — ~~starts_on.transform_env(out_sndr, env) is equivalent to:~~

```
~~auto&& [_, sch, _] = out_sndr;~~
~~return JOIN-ENV(SCHED-ENV(sch), FWD-ENV(env));~~
```

(4.2) — starts_on.transform_sender(set_value,out_sndr, env) is equivalent to:

```
auto&& [_, sch, sndr] = out_sndr;
return let_value(
  ~~schedule(sch),~~
  continues_on(just(), sch),
    [sndr = std::forward_like<OutSndr>(sndr)]()
        mutable
     noexcept(is_nothrow_move_constructible_v<de-
        cay_t<OutSndr>>) {
    return std::move(sndr);
  });
```

5. The exposition-only class template *impls-for* is specialized for starts_on_t as follows:

```
namespace std::execution {
  template<>
   struct impls-for<starts_on_t> : default-impls
        {
    static constexpr auto get-attrs =
        [](const auto& sch, const auto& child)
        noexcept -> decltype(auto) {
        return see below;
      };
  };
}
```

For subexpressions sch and child and pack of subexpressions env, let sch2 be get_completion_scheduler<set_value_t>(sch, env...), and let env2 be the pack *JOIN-ENV*(*SCHED-ENV*(sch2), env). *impls-for*<starts_on_t>::*get-attrs*(sch, child) returns an object attrs such that

(5.1) — `attrs.query(get_completion_scheduler<completion-tag>, env...)` is expression-equivalent to `get_completion_scheduler<completion-tag>(get_env(child), env2...)`.

(5.2) — `attrs.query(get_completion_domain<completion-tag>, env...)` is expression-equivalent to `get_completion_domain<completion-tag>(get_env(child), env2...)`.

(5.3) — For a subexpression `q` and pack `args`, `attrs.query(q, args...)` is expression-equivalent to *FWD-ENV*`(get_env(child)).query(q, args...)` where the type of `q` is not a specialization of either `get_completion_scheduler_t` or `get_completion_domain_t`.

[Editor's note: Remove subsection 33.9.12.6 [exec.continues.on]]

[Editor's note: Change stable name 33.9.12.7 [exec.schedule.from] to [exec.continues.on], and change the subsection as follows:]

33.9.12.~~7~~6 `execution::`~~`schedule_from`~~`continues_on` [exec.~~sched-ule.from~~.continues.on]

1. ~~`schedule_from`~~`continues_on` schedules work dependent on the completion of a sender onto a scheduler's associated execution resource.

   ~~[*Note 1*: `schedule_from` is not meant to be used in user code; it is used in the implementation of `continues_on`. — end note]~~

2. The name ~~`schedule_from`~~`continues_on` denotes a customization point object. For some subexpressions `sch` and `sndr`, let `Sch` be `decltype((sch))` and `Sndr` be `decltype((sndr))`. If `Sch` does not satisfy scheduler, or `Sndr` does not satisfy `sender`, ~~`schedule_from(sch, sndr)`~~`continues_on(sndr, sch)` is ill-formed.

3. Otherwise, the expression ~~`schedule_from(sch, sndr)`~~`continues_on(sndr, sch)` is expression-equivalent to~~:~~ *`make-sender`*`(continues_on, sch, schedule_from(sndr))`

   ~~`transform_sender(`~~
   ~~`query-with-default(get_domain, sch, default_domain()),`~~
   ~~`make-sender(schedule_from, sch, sndr))`~~

   ~~except that sch is evaluated only once.~~

4. The exposition-only class template *`impls-for`* (33.9.1 [exec.snd.general]) is specialized for ~~`schedule_from_t`~~`continues_on_t` as follows:

```
namespace std::execution {
   template<>
        struct  impls-for<~~schedule_from_t~~continues_on_t> : default-impls {
         static constexpr auto get-attrs = see below;
         static constexpr auto get-state = see below;
          static constexpr auto complete = see below;

      template<class Sndr, class... Env>
        static consteval void check-types();
   };
}
```

5. The member *impls-for*<~~schedule_from_t~~continues_on_t>::*get-attrs* is initialized with a callable object equivalent to the following lambda:

```
[](const auto& data, const auto& child) noexcept -> decltype(auto) {
      ~~return JOIN-ENV(SCHED-ATTRS(data), FWD-ENV(get_env(child)));~~
  return SCHED-ATTRS(data, child);
}
```

6. The member *impls-for*<~~schedule_from_t~~continues_on_t>::*get-state* is initialized with a callable object equivalent to the following lambda:

   … as before …

… as before …

12. The member *impls-for*<~~schedule_from_t~~continues_on_t>::*complete* is initialized with a callable object equivalent to the following lambda:

    … as before …

13. Let `out_sndr` be a subexpression denoting a sender returned from ~~`schedule_from(sch, sndr)`~~`continues_on(sndr, sch)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be *… as before …*

[Editor's note: After 33.9.12.6 [exec.continues.on], add a new subsection with stable name [exec.schedule.from] as follows:]

§ `execution::schedule_from` [exec.schedule.from]

1. `schedule_from` offers scheduler authors a way to customize how to transition off of their schedulers' associated execution contexts.

   [*Note:* `schedule_from` is not meant to be used in user code; it is used in the implementation of `continues_on`. — *end note*]

2. The name `schedule_from` denotes a customization point object. For some subexpression `sndr`, if `decltype(sndr)` does not satisfy `sender`, `schedule_from(sndr)` is ill-formed.

3. Otherwise, the expression `schedule_from(sndr)` is expression-equivalent to *`make-sender`*`(schedule_from, {}, sndr)`.

[Editor's note: Change 33.9.12.8 [exec.on] as follows:]

§ `execution::on` [exec.on]

1. The `on` sender adaptor has two forms *…as before …*

2. The name `on` denotes a *…as before …*

3. Otherwise, if `decltype((sndr))` satisfies `sender`, the expression `on(sch, sndr)` is expression-equivalent to:

```
~~transform_sender(~~
  ~~query-with-default(get_domain, sch, default_domain()),~~
  make-sender(on, sch, sndr)~~)~~
```

   ~~except that `sch` is evaluated only once.~~

4. For subexpressions `sndr`, `sch`, and `closure`, if

(4.1) — `decltype((sch))` does not satisfy `scheduler`, or

(4.2) — `decltype((sndr))` does not satisfy `sender`, or

(4.3) — `closure` is not a pipeable sender adaptor closure object (33.9.12.2 [exec.adapt.obj]),

   the expression `on(sndr, sch, closure)` is ill-formed; otherwise, it is expression-equivalent to:

```
~~transform_sender(~~
  ~~get-domain-early(sndr),~~
  make-sender(on, product-type{sch, closure}, sndr)~~)~~
```

   ~~except that `sndr` is evaluated only once.~~

5. Let `out_sndr` and `env` be subexpressions, let `OutSndr` be `decltype((out_sndr))`, and let `Env` be `decltype((env))`. If *`sender-for`*`<OutSndr, on_t>` is `false`, then the expression~~`s-on.transfor-`~~

~~m_env(out_sndr, env) and~~ `on.transform_sender(set_value,out_sndr, env)` ~~are~~is ill-formed.

6. ~~Otherwise: Let *not-a-scheduler* be an unspecified empty class type.~~

7. ~~The expression `on.transform_env(out_sndr, env)` has effects equivalent to:~~

```
auto&& [_, data, _] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
  return JOIN-ENV(SCHED-ENV(std::forward_like<OutSndr>(data)),
    FWD-ENV(std::forward<Env>(env)));
} else {
  return std::forward<Env>(env);
}
```

8. Otherwise, ~~T~~the expression `on.transform_sender(set_value,out_sndr, env)` has effects equivalent to:

```
auto&& [_, data, child] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
  auto orig_sch =
    query-with-default(get_scheduler, env, not-a-scheduler());

  if constexpr (same_as<decltype(orig_sch), not-a-scheduler>) {
    return not-a-sender{};
  } else {
    return continues_on(
      starts_on(std::forward_like<OutSndr>(data), std::forward_like<OutSndr>(child)),
      std::move(orig_sch));
  }
} else {
  auto& [sch, closure] = data;
  auto orig_sch = query-with-default(
    get_completion_scheduler<set_value_t>,
    get_env(child),
    query-with-default(get_scheduler, env, not-a-scheduler()));

  if constexpr (same_as<decltype(orig_sch), not-a-scheduler>) {
    return not-a-sender{};
  } else {
    return write_env(
      continues_on(
        std::forward_like<OutSndr>(closure)(
          continues_on(
            write_env(std::forward_like<OutSndr>(child), SCHED-ENV(orig_sch)),
            sch)),
```

```
~~        orig_sch),~~
~~      SCHED_ENV(sch));~~
~~  }~~
~~}~~
```

```
auto&& [_, data, child] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
  auto orig_sch =
        call-with-default(get_scheduler,  not-a-
        scheduler(), env);

  return continues_on(
      starts_on(std::forward_like<OutSndr>(data),
        std::forward_like<OutSndr>(child)),
    std::move(orig_sch));
} else {
  auto& [sch, closure] = data;
  auto orig_sch = call-with-default(
      get_completion_scheduler<set_value_t>, not-
        a-scheduler(), get_env(child), env);

  return continues_on(
      std::forward_like<OutSndr>(closure)(contin-
        ues_on(std::forward_like<OutSndr>
        (child), sch)),
    orig_sch);
}
```

9. The exposition-only class template *impls-for* (33.9.2 [exec.snd.expos]) is specialized for `on_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<on_t> : default-impls {
    static constexpr auto get-attrs =
        [](const auto& data, const auto& child)
        noexcept {
        return see below;
      };
  };
}
```

Given a pack of subexpressions `env`, let `Env` be the pack `decltype(env)`, let `Data` be the decayed type of `data`, let `sndr` be a `const` lvalue reference to the *basic-sender* object that owns the objects to which `data` and `child` refer, and let `new_attrs` be a *queryable* object equal to `get_env(on.transform_sender(set_value, sndr_copy, env...))` where `sndr_copy` is a hypothetical non-`const` object equal to `sndr`. The above lambda returns an object `attrs` such that, for a query object `q` and pack of subexpressions `as`, `attrs.query(q, as...)` is ill-formed if `new_attrs.query(q, as...)` is ill-formed; otherwise, it has the same type and value as `new_attrs.query(q, as...)`.

Recommended practice: Implementations should compute the results of queries without actually calling `on_t::transform_sender`.

[Editor's note: Change 33.9.12.9 [exec.then] paragraphs 3 and 4 and add a new paragraph as follows:]

3. Otherwise, the expression *then-cpo*`(sndr, f)` is expression-equivalent to:

```
~~transform_sender(get-domain-early(sndr),~~    make-
        sender(then-cpo, f, sndr)~~)~~
```

~~except that `sndr` is evaluated only once.~~

4. For `then`, `upon_error`, and `upon_stopped`, let *set-cpo* be `set_value`, `set_error`, and `set_stopped`, respectively. The exposition-only class template *impls-for* (33.9.2 [exec.snd.expos]) is specialized for *then-cpo* as follows:

```
namespace std::execution {
  template<>
   struct  impls-for<decayed-typeof<then-cpo>>  :
        default-impls {
    static constexpr auto get-attrs =
       [](const auto& data, const auto& child) {
        return see below; };

    static constexpr auto complete = … as before
        …;

    template<class Sndr, class... Env>
      static consteval void check-types();
  };
}
```

5. Let `q` be a query object and let `as` be a pack of subexpressions. For subexpressions `child` and `fn`, *impls-for*`<decayed-typeof<then-cpo>>::get-attrs(fn, child)` returns an object `o` whose type satisfies *queryable* such that `o.query(q, as...)` is expression-equivalent to:

(5.1) — `D()` if `q` is `get_completion_domain<set_value_t>`, where `D` is the type of the expression

```
COMMON-DOMAIN(COMPL-DOMAIN(decayed-type-
        of<set-cpo>, child, FWD-ENV(as)...),
                    COMPL-DOMAIN(set_value_t,
        child, FWD-ENV(as)...))
```

(5.2) — `D()` if `q` is `get_completion_domain<set_error_t>` and *gather-signatures*`<decayed-typeof<set-cpo>, CS, is-nothrow, conjunction>::value` is `false`, where `CS` is `completion_signa-`

tures_of_t<decay_t<decltype((child))>, *FWD-ENV-T*(decltype((as)))...>; *is-nothrow* is an alias template such that for a pack of types Ts, *is-nothrow*<Ts...> is is_nothrow_invocable<decay_t<decltype((fn))>, Ts...>; and D is the type of the expression

```
COMMON-DOMAIN(COMPL-DOMAIN(decayed-type-
        of<set-cpo>, child, FWD-ENV(as)...),
                    COMPL-DOMAIN(set_error_t,
        child, FWD-ENV(as)...))
```

(5.3) — Otherwise, *FWD-ENV*(get_env(child)).query(q, as...) if either of the following is true:

(5.2.1) — q is get_completion_scheduler<set_value_t> and *set-cpo* is set_value, or

(5.2.2) — q is not get_completion_domain<*decayed-typeof*<*set-cpo*>> or get_completion_scheduler<*decayed-typeof*<*set-cpo*>>.

(5.4) — Otherwise, o.query(q, as...) is ill-formed.

[Editor's note: Change 33.9.12.10 [exec.let] as follows:]

1. let_value, let_error, and let_stopped *… as before …*

2. For let_value, let_error, and let_stopped, let *set-cpo* be set_value, set_error, and set_stopped, respectively. Let the expression *let-cpo* be one of let_value, let_error, or let_stopped. For ~~a~~ subexpressions sndr and env, let *let-env*(sndr, env) be expression-equivalent to the first well-formed expression below:

(2.1) — *SCHED-ENV*(get_completion_scheduler<*decayed-typeof*<*set-cpo*>>(get_env(sndr), *FWD-ENV*(env)))

~~(2.2)~~ — ~~*MAKE-ENV*(get_domain, get_domain(get_env(sndr)))~~

(2.2) — *MAKE-ENV*(get_domain, get_completion_domain<*decayed-typeof*<*set-cpo*>>(get_env(sndr), *FWD-ENV*(env)))

(2.3) — (void(sndr), env<>{})

3. The names let_value, let_error, and let_stopped denote *… as before …*

4. Otherwise, the expression *let-cpo*(sndr, f) is expression-equivalent to:

```
~~transform_sender(get-domain-early(sndr),~~    make-
        sender(let-cpo, f, sndr)~~)~~
```

~~except that sndr is evaluated only once.~~

5. The exposition-only class template *impls-for* (33.9.2 [exec.snd.expos]) is specialized for *let-cpo* as follows:

```
namespace std::execution {
   template<class State, class Rcvr, class...
       Args>
    void let-bind(State& state, Rcvr& rcvr,
       Args&&... args);      // exposition only

  template<>
    struct impls-for<decayed-typeof<let-cpo>> :
       default-impls {
    static constexpr auto get-attrs = see below;
    static constexpr auto get-state = see below;
    static constexpr auto complete = see below;

    template<class Sndr, class... Env>
      static consteval void check-types();
  };
}
```

6. Let *receiver2* denote the following exposition-only class template:

```
namespace std::execution {
  template<class Rcvr, class Env>
  struct receiver2 {
    … as before …
  };
}
```

Invocation of the function *receiver2*::get_env returns an object e such that

(6.1) — decltype(e) models *queryable* and

(6.2) — given a query object q and pack of subexpressions as, the expression e.query(q, as...) is expression-equivalent to env.query(q, as...) if that expression is valid; otherwise, if the type of q satisfies *forwarding-query*, e.query(q, as...) is expression-equivalent to get_env(rcvr).query(q, as...); otherwise, e.query(q, as...) is ill-formed.

7. *Effects:* Equivalent to:

… as before …

where *env-t* is the pack ~~decltype(*let-cpo*.transform_env(declval<Sndr>(), declval<Env>()))~~ decltype(*JOIN-ENV*(*let-env*(declval<*child-type*<Sndr>>(), declval<Env>()), *FWD-ENV*(declval<Env>())))

8. *impls-for*<*decayed-typeof*<*let-cpo*>>::*get-attrs* is initialized with a callable object equivalent to the following:

```
[](const auto& fn, const auto& child) {
  return see below;
}
```

Let SetCPO be the decayed type of *set-cpo*, let Fn be the decayed type of fn, and let Sndrs be a pack of the parameters of the tuple specialization named by *gather-signatures*<SetCPO, CS, *invoke-result*, tuple>, where CS is completion_signatures_of_t<decay_t<decltype((child))>, *FWD-ENV-T*(decltype((as)))...> and *invoke-result* is an alias template such that *invoke-result*<Ts...> is invoke_result_t<Fn, Ts...> for a pack Ts. The lambda above returns an object o whose type satisfies *queryable* such that, for a query object q and pack of subexpressions as, let let_env be the pack *JOIN-ENV*(*let-env*(child, as), *FWD-ENV*(as)). Then o.query(q, as...) is as follows:

(8.1) — If q is get_completion_domain<SetCPO>, o.query(q, as...) is expression-equivalent to D() where D is the type of the expression:

```
COMMON-DOMAIN(COMPL-DOMAIN(SetCPO,
        declval<Sndrs>(), let_env...)...)
```

(8.2) — Otherwise, if q is get_completion_domain<set_error_t> and *gather-signatures*<SetCPO, CS, *is-nothrow*, conjunction>::value is well-formed and false, o.query(q, as...) is ill-formed if sizeof...(as) == 0 is true; otherwise it is expression-equivalent to D(), where *is-nothrow* is the alias template

```
template<class... Ts>
  using is-nothrow = bool_constant<
    (noexcept(auto(declval<Ts>())) &&...) &&
      noexcept(connect(invoke(declval<Fn>(),
        declval<decay_t<Ts>&>()...),
                              receiver-arche-
        type())>;
```

where *receiver-archetype* is an empty class type such that receiver_of<*receiver-archetype*, U> is true for any specialization of completion_signatures U and such that the type of *receiver-archetype*().get_env() is decltype((as...[0])) [Editor's note: This assumes the adoption of [P3388R2], forwarded to LWG during the Kona 2025 meeting.]; and where D is the type of the following expression:

```
COMMON-DOMAIN(COMPL-DOMAIN(SetCPO,     child,
        FWD-ENV(as)...),
                    COMPL-DOMAIN(set_error_t,
        child, FWD-ENV(as)...),
                   COMPL-DOMAIN(set_error_t, de-
        clval<Sndrs>(), let_env...)...)
```

(8.3) — Otherwise, if `q` is `get_completion_domain<set_error_t>` or `get_completion_domain<set_stopped_t>`, let `Tag` be the template parameter of the type of `q`. Then `o.query(q, as...)` is `D()` where `D()` is the type of the following expression:

```
COMMON-DOMAIN(COMPL-DOMAIN(Tag,  child,  FWD-
        ENV(as)...),
                              COMPL-DOMAIN(Tag,
        declval<Sndrs>(), let_env...)...)
```

8. *impls-for*<*decayed-typeof*<*let-cpo*>>::*get-state* is initialized with a callable object equivalent to the following:

```
[]<class  Sndr,  class  Rcvr>(Sndr&&  sndr,  Rcvr&
        rcvr) requires see below {
  auto& [_, fn, child] = sndr;
  using fn_t = decay_t<decltype(fn)>;
      using   env_t   =   decltype(let-env(child,_
        get_env(rcvr)));
  using args_variant_t = see below;
  using ops2_variant_t = see below;

  struct state-type {
    fn_t fn;                          // exposition
        only
    env_t env;                        // exposition
        only
     args_variant_t args;             // exposition
        only
     ops2_variant_t ops2;             // exposition
        only
  };
        return    state-type{allocator-aware-for-
        ward(std::forward_like<Sndr>(fn), rcvr),
                                       let-env(child,_
        get_env(rcvr)), {}, {}};
}
```

[Editor's note: leave paragraphs 9-13 unchanged]

14. ~~Let `sndr` and `env` be subexpressions, and let `Sndr` be `decltype((sndr))`. If *sender-for*<Sndr, *decayed-typeof*<*let-cpo*>> is `false`, then the expression *let-cpo*`.transform_env(sndr, env)` is ill-formed. Otherwise, it is equal to:~~

```
auto& [_, _, child] = sndr;
return JOIN-ENV(let-env(child), FWD-ENV(env));
```

15. Let the subexpression `out_sndr` denote *… as before …*

[Editor's note: Change 33.9.12.11 [exec.bulk] paragraph 3 and 4 as follows:]

3. Otherwise, the expression *`bulk-algo`*`(sndr, policy, shape, f)` is expression-equivalent to:

```
~~transform_sender(get-domain-early(sndr),~~   make-sender(
    bulk-algo, product-type<see below, Shape, Func>{policy, shape, f}, sndr)~~)~~
```

~~except that `sndr` is evaluated only once.~~

The first template argument of *`product-type`* is `Policy` if `Policy` models `copy_constructible`, and `const Policy&` otherwise.

4. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))`. If *`sender-for`*`<Sndr, bulk_t>` is `false`, then the expression `bulk.transform_sender(set_value, sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto [_, data, child] = sndr;
auto& [policy, shape, f] = data;
auto new_f = [func = std::move(f)](Shape begin, Shape end,
        auto&&... vs)
    noexcept(noexcept(f(begin, vs...))) {
  while (begin != end) func(begin++, vs...);
}
return   bulk_chunked(std::move(child),   policy,   shape,
        std::move(new_f));
```

[*Note:* This causes the `bulk(sndr, policy, shape, f)` sender to be expressed in terms of `bulk_chunked(sndr, policy, shape, f)` when it is connected to a receiver whose execution domain does not customize `bulk`. — *end note*]

[Editor's note: Change 33.9.12.12 [exec.when.all] paragraphs 2 and 3 as follows:]

2. The names `when_all` and `when_all_with_variant` denote customization point objects. Let `sndrs` be a pack of subexpressions~~,~~ and let `Sndrs` be a pack of the types `decltype((sndrs))...` ~~, and let CD be the type `common_type_t<decltype(get-domain-early(sndrs))...>`. Let CD2 be CD if CD is well-formed, and `default_domain` otherwise~~. The expressions `when_all(sndrs...)` and `when_all_with_variant(sndrs...)` are ill-formed if any of the following is `true`:

(2.1) — `sizeof...(sndrs)` is `0`, or

(2.2) — `(sender<Sndrs> && ...)` is `false`.

3. The expression `when_all(sndrs...)` is expression-equivalent to:

```
~~transform_sender(CD2(),~~     make_sender(when_all,
        {}, sndrs...)~~)~~
```

[Editor's note: Change 33.9.12.12 [exec.when.all] paragraphs 9 and 10 as follows:]

```
template<class Sndr, class... Env>
  static consteval void check_types();
```

7. Let `Is` be the pack of integral template arguments of the `integer_sequence` specialization denoted by *`indices-for`*`<Sndr>`.

8. *Effects*: Equivalent to: *… as before …*

9. ~~*Throws*: Any exception thrown as a result of evaluating the *Effects*, or an exception of an unspecified type derived from `exception` when `CD` is ill-formed.~~

10. The member *`impls-for`*`<when_all_t>::`*`get-attrs`* is initialized with a callable object equivalent to the following lambda expression:

```
[](auto&&, auto&&... child) noexcept {
  ~~if constexpr (same_as<CD, default_domain>) {~~
    ~~return env<>();~~
  ~~} else {~~
    ~~return MAKE_ENV(get_domain, CD());~~
  ~~}~~
  return see below;
}
```

Let *`WHEN-ALL-DOMAIN`*`(Tag, env...)` be the expression *`COMMON-DOMAIN`*`(`*`COMPL-DOMAIN`*`(Tag, child, env...)...)`. The lambda expression above returns an object `attrs` such that

(10.1) — `attrs.query(get_completion_domain<set_value_t>, env...)` is expression-equivalent to *`WHEN-ALL-DOMAIN`*`(set_value_t, env...)`.

(10.2) — `attrs.query(get_completion_domain<set_error_t>, env...)` is expression-equivalent to:

```
COMMON-DOMAIN(WHEN-ALL-DOMAIN(set_value_t,
        env...),
                WHEN-ALL-DOMAIN(set_error_t,
        env...),
              WHEN-ALL-DOMAIN(set_stopped_t,
        env...))
```

(10.3) — `attrs.query(get_completion_domain<set_stopped_t>, env...)` is expression-equivalent to: [Editor's note: This assumes the adoption of **P3887R0?**, which was forwarded to LWG at the Kona 2025 meeting.]

```
COMMON-DOMAIN(WHEN-ALL-DOMAIN(set_value_t,
        env...),
              WHEN-ALL-DOMAIN(set_stopped_t,
        env...))
```

[Editor's note: Change 33.9.12.12 [exec.when.all] paragraphs 19 and 20 as follows:]

19. The expression `when_all_with_variant(sndrs...)` is expression-equivalent to:

```
~~transform_sender(CD2(),~~      make-sender(when_al-
        l_with_variant, {}, sndrs...)~~);~~
```

20. Given subexpressions `sndr` and `env`, if *sender-for*<`decltype((sndr))`, `when_all_with_variant_t`> is `false`, then the expression `when_all_with_variant.transform_sender(set_value, sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, _, ...child] = sndr;
return       when_all(into_variant(std::forward_-
        like<decltype((sndr))>(child))...);
```

[*Note:* This causes the `when_all_with_variant(sndrs...)` sender to become `when_all(into_variant(sndrs)...)` when it is connected with a receiver whose execution domain does not customize `when_all_with_variant`. — *end note*]

[Editor's note: Change 33.9.12.13 [exec.into.variant] paragraph 3 as follows:]

3. Otherwise, the expression `into_variant(sndr)` is expression-equivalent to:

```
~~transform_sender(get-domain-early(sndr),~~
                    make-sender(into_variant, {},
        sndr)~~);~~
```

[Editor's note: Change 33.9.12.14 [exec.stopped.opt] paragrpah 2-4 as follows:]

2. The name `stopped_as_optional` denotes a pipeable sender adaptor object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `stopped_as_optional(sndr)` is expression-equivalent to:

```
~~transform_sender(get-domain-early(sndr),~~
                    make-sender(stopped_as_option-
        al, {}, sndr)~~)~~
```

~~except that `sndr` is only evaluated once.~~

3. The exposition-only class template *impls-for* (33.9.2 [exec.snd.expos]) is specialized for `stopped_as_optional_t` as follows:

```
namespace std::execution {
  template<>
   struct impls-for<stopped_as_optional_t> : de-
        fault-impls {
    static constexpr auto get-attrs =
        [](const auto&, const auto& child) noex-
        cept -> decltype(auto) {
        see below
      };

    template<class Sndr, class... Env>
      static consteval void check-types() {
        … as before …
      }
  };
}
```

where *unspecified-exception* is a type derived from `exception`.

4. For subexpressions `ign` and `child`, pack of subexpressions `as`, and query object `q`, *impls-for*`<stopped_as_optional_t>::`*get-attrs*`(ign, child)` returns an object `o` such that `o.query(q, as...)` is expression-equivalent to:

(4.1) — `D()` if `q` is `get_completion_domain<set_value_t>`, where `D` is the type of the expression

```
COMMON-DOMAIN(COMPL-DOMAIN(set_value_t,
        child, FWD-ENV(as)...),
                 COMPL-DOMAIN(set_stopped_t,
        child, FWD-ENV(as)...))
```

(4.2) — *FWD-ENV*`(get_env(child)).query(q, as...)` if `q` is not `get_completion_domain<set_stopped_t>`, `get_completion_scheduler<set_value_t>`, or `get_completion_scheduler<set_stopped_t>`.

(4.3) — Otherwise, `o.query(q, as...)` is ill-formed.

5. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If *sender-for*`<Sndr, stopped_as_optional_t>` is `false` then the expression `stopped_as_optional.transform_sender(set_value, sndr, env)` is ill-formed; otherwise, if `sender_in<`*child-type*`<Sndr>, `*FWD-ENV-T*`(Env)>` is `false`, the expression `stopped_as_optional.transform_sender(set_value, sndr, env)` is equivalent to *not-a-sender*`()`; otherwise, it is equivalent to:

```
auto&& [_, _, child] = sndr;
using    V    =    single-sender-value-type<child-
        type<Sndr>, FWD-ENV-T(Env)>;
return let_stopped(
  then(std::forward_like<Sndr>(child),
```

```
            []<class... Ts>(Ts&&... ts) noex-
        cept(is_nothrow_constructible_v<V,
        Ts...>) {
          return optional<V>(in_place, std::for-
        ward<Ts>(ts)...);
      }),
    []() noexcept { return just(optional<V>());
        });
```

[Editor's note: Change 33.9.12.15 [exec.stopped.err] paragraphs 2-3 as follows:]

2. The name `stopped_as_error` denotes a pipeable sender adaptor object. For some subexpressions `sndr` and `err`, let `Sndr` be `decltype((sndr))` and let `Err` be `decltype((err))`. If the type `Sndr` does not satisfy `sender` or if the type `Err` does not satisfy *`movable-value`*, `stopped_as_error(sndr, err)` is ill-formed. Otherwise, the expression `stopped_as_error(sndr, err)` is expression-equivalent to:

```
~~transform_sender(get-domain-early(sndr),~~    make-
        sender(stopped_as_error, err, sndr)~~)~~
```

~~except that `sndr` is only evaluated once.~~

3. The exposition-only class template *`impls-for`* (33.9.2 [exec.snd.expos]) is specialized for `stopped_as_error_t` as follows:

```
namespace std::execution {
  template<>
    struct impls-for<stopped_as_error_t> : de-
        fault-impls {
    static constexpr auto get-attrs =
        [](const auto&, const auto& child) noex-
        cept -> decltype(auto) {
        see below
      };
  };
}
```

4. For subexpressions `ign` and `child`, pack of subexpressions `as`, and query object `q`, *`impls-for`*`<stopped_as_error_t>::`*`get-attrs`*`(ign, child)` returns an object `o` such that `o.query(q, as...)` is expression-equivalent to:

(4.1) — `D()` if `q` is `get_completion_domain<set_error_t>`, where `D` is the type of the expression

```
COMMON-DOMAIN(COMPL-DOMAIN(set_error_t,
        child, FWD-ENV(as)...),
                COMPL-DOMAIN(set_stopped_t,
        child, FWD-ENV(as)...))
```

(4.2) — *`FWD-ENV`*`(get_env(child)).query(q, as...)` if `q` is not `get_completion_domain<set_stopped_t>`, `get_completion_sched-`

uler<set_error_t>, or get_completion_scheduler<set_stopped_t>.

(4.3) — Otherwise, `o.query(q, as...)` is ill-formed.

5. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If *`sender-for`*`<Sndr, stopped_as_error_t>` is `false`, then the expression `stopped_as_error.transform_sender(set_value, sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, err, child] = sndr;
using E = decltype(auto(err));
return let_stopped(
  std::forward_like<Sndr>(child),
   [err = std::forward_like<Sndr>(err)]() mutable noexcept(is_nothrow_move_constructible_v<E>) {
    return just_error(std::move(err));
  });
```

[Editor's note: Change 33.9.12.16 [exec.associate] paragraph 10 as follows:]

10. The name `associate` denotes a pipeable sender adaptor object. For subexpressions `sndr` and `token`:

(10.1) — If `decltype((sndr))` does not satisfy `sender`, or `remove_cvref_t<decltype((token))>` does not satisfy `scope_token`, then `associate(sndr, token)` is ill-formed.

(10.2) — Otherwise, the expression `associate(sndr, token)` is expression-equivalent to:

```
~~transform_sender(get-domain-early(sndr),~~
                    make-sender(associate, associate-data(token, sndr))~~)~~
```

~~except that `sndr` is evaluated only once.~~

[Editor's note: Change 33.9.13.1 [exec.sync.wait] paragraphs 4-9 as follows:]

4. The name `this_thread::sync_wait` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `this_thread::sync_wait(sndr)` is ~~expression-~~equivalent to the following, except that `sndr` is evaluated only once:

```
run_loop loop;
sync-wait-env env{&loop};
auto domain = get_completion_domain<set_value_t>
        (get_env(sndr), env);
return     apply_sender(~~get-domain-early(sndr)~~domain, sync_wait, sndr, env);
```

Mandates:

(4.1) — `sender_in<Sndr, sync-wait-env>` is `true`.

(4.2) — The type *`sync-wait-result-type`*`<Sndr>` is well-formed.

(4.3) — `same_as<decltype(e), sync-wait-result-type<Sndr>>` is `true`, where *e* is the `apply_sender` expression above.

5. Let *`sync-wait-state`* and *`sync-wait-receiver`* be the following exposition-only class templates: [Editor's note: Note the addition of & to the declaration of the *`sync-wait_state`*`::loop` data member.]

```
namespace std::this_thread {
  template<class Sndr>
            struct       sync-wait-state       {
        // exposition only
                    execution::run_loop&     loop;
        // exposition only
                      exception_ptr      error;
        // exposition only
            sync-wait-result-type<Sndr>   result;
        // exposition only
  };

  template<class Sndr>
            struct      sync-wait-receiver      {
        // exposition only
       using  receiver_concept  =  execution::re-
        ceiver_t;
                sync-wait-state<Sndr>*    state;
        // exposition only

    template<class... Args>
    void set_value(Args&&... args) && noexcept;

    template<class Error>
    void set_error(Error&& err) && noexcept;

    void set_stopped() && noexcept;

     sync-wait-env get_env() const noexcept { re-
         turn {&state->loop}; }
  };
}
```

[Editor's note: Leave paragraphs 6-8 as-is]

9. For ~~a~~ subexpressions `sndr` and `env`, let `Sndr` be `decltype((sndr))`. If `sender_to<Sndr, sync-wait-receiver<Sndr>>` is `false` or if the type of `env` is not *`sync-wait-env`*, the expression `sync_wait.apply_sender(sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
sync-wait-state<Sndr> state{*env.loop};
auto op = connect(sndr, sync-wait-receiver<Sndr>
        {&state});
start(op);

state.loop.run();
if (state.error) {
  rethrow_exception(std::move(state.error));
}
return std::move(state.result);
```

[Editor's note: Change 33.9.13.2 [exec.sync.wait.var] as follows:]

1. The name `this_thread::sync_wait_with_variant` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype(into_variant(sndr))`. The expression `this_thread::sync_wait_with_variant(sndr)` is ~~expression-~~equivalent to the following, except `sndr` is evaluated only once:

```
run_loop loop;
sync-wait-env env{&loop};
auto domain = get_completion_domain<set_value_t>
        (get_env(sndr), env);
return    apply_sender(~~get-domain-early(sndr)~~do-
        main,   sync_wait_with_variant,   sndr,
        env);
```

Mandates:

(1.1) — `sender_in<Sndr, sync-wait-env>` is `true`.

(1.2) — The type *`sync-wait-with-variant-result-type`*`<Sndr>` is well-formed.

(1.3) — `same_as<decltype(e), sync-wait-with-variant-result-type<Sndr>>` is `true`, where *`e`* is the `apply_sender` expression above.

2. For subexpressions `sndr` and `env`, let `Sndr` be `decltype(into_variant(sndr))`. If `sender_to<Sndr, sync-wait-receiver<Sndr>>` is `false` or if the type of `env` is not *`sync-wait-env`*, ~~T~~tthe expression `sync_wait_with_variant.apply_sender(sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
using result_type = sync-wait-with-variant-re-
        sult-type<Sndr>;
if (auto opt_value = sync_wait.apply_sender(in-
        to_variant(sndr), env)) {
   return result_type(std::move(get<0>(*opt_val-
        ue)));
}
return result_type(nullopt);
```

3. The behavior of `this_thread::sync_wait_with_variant(sndr)` is undefined unless *… as before …*

[Editor's note: Change 33.11.1 [exec.prop] as follows:]

```
namespace std::execution {
  template<class QueryTag, class ValueType>
  struct prop {
    QueryTag query_;            // exposition only
    ValueType value_;           // exposition only

    constexpr const ValueType& query(QueryTag, auto&&...) const
        noexcept {
      return value_;
    }
  };

  template<class QueryTag, class ValueType>
            prop(QueryTag,    ValueType)    ->    prop<QueryTag,
        unwrap_reference_t<ValueType>>;
}
```

… as before …

[Editor's note: Change 33.11.2 [exec.env] as follows:]

```
namespace std::execution {
  template<queryable... Envs>
  struct env {
    Envs_0 envs_0;              // exposition only
    Envs_1 envs_1;              // exposition only
      ⋮
    Envs_n-1 envs_n-1;          // exposition only

    template<class QueryTag, class... Args>
          constexpr decltype(auto) query(QueryTag q,
        Args&&... args) const noexcept(see below);
  };

  template<class... Envs>
    env(Envs...) -> env<unwrap_reference_t<Envs>...>;
}
```

1. The class template `env` is used to construct a queryable object from several queryable objects. Query invocations on the resulting object are resolved by attempting to query each subobject in lexical order.

   … as before …

```
template<class QueryTag, class... Args>
constexpr decltype(auto) query(QueryTag q, Args&&...
        args) const noexcept(see below);
```

5. Let *has-query* be the following exposition-only concept:

```
template<class Env, class QueryTag, class... Args>
  concept has-query =                    // exposition only
    requires (const Env& env, Args&&... args) {
      env.query(QueryTag(), std::forward<Args>(args)...);
    };
```

6. Let *fe* be the first element of $envs_0$, $envs_1$, … $envs_{n-1}$ such that the expression `fe.query(q, std::forward<Args>(args)...)` is well-formed.

7. *Constraints*: `(has-query<Envs, QueryTag, Args...> || ...)` is `true`.

8. *Effects*: Equivalent to: `return fe.query(q, std::forward<Args>(args)...);`

9. *Remarks*: The expression in the `noexcept` clause is equivalent to `noexcept(fe.query(q, std::forward<Args>(args)...))`.

[Editor's note: In 33.12.1.2 [exec.run.loop.types], add a new paragraph after paragraph 4 as follows:]

4. Let *sch* be an expression of type *run-loop-scheduler*. The expression `schedule(sch)` has type *run-loop-sender* and is not potentially-throwing if *sch* is not potentially-throwing.

5. For type *set-tag* other than `set_error_t`, the expression `get_completion_scheduler<set-tag>(get_env(schedule(sch))) == sch` evaluates to `true`.

[Editor's note: Change 33.13.3 [exec.affine.on] paragraphs 3 and 4 as follows:]

3. Otherwise, the expression `affine_on(sndr, sch)` is expression-equivalent to~~:~~ *make-sender*`(affine_on, sch, sndr)`.

~~`transform_sender(get-domain-early(sndr), make-sender(affine_on, sch, sndr))`~~

~~except that `sndr` is evaluated only once.~~

4. The exposition-only class template *impls-for* (33.9.2 [exec.snd.expos]) is specialized for `affine_on_t` as follows:

```
namespace std::execution {
  template<>
   struct impls-for<affine_on_t> : default-impls {
    static constexpr auto get-attrs =
```

```
        [](const auto& data, const auto& child)
        noexcept -> decltype(auto) {
          return JOIN-ENV(SCHED-ATTRS(data), FWD-ENV(get_env(child)));
        return SCHED-ATTRS(data, child);
      };
  };
}
```

[Editor's note: Change 33.13.4 [exec.inline.scheduler] paragraphs 1-3 as follows:]

1. `inline_scheduler` is a class that models `scheduler` (33.6 [exec.sched]). All objects of type `inline_scheduler` are equal. For a subexpression `sch` of type `inline_scheduler`, a query object `q`, and a pack of subexpressions `as`, the expression `sch.query(q, as...)` is expression-equivalent to *`inline-attrs`*`<set_value_t>().query(q, as...)`.

2. *`inline-sender`* is an exposition-only type that satisfies `sender`. The type `completion_signatures_of_t<`*`inline-sender`*`>` is `completion_signatures<set_value_t()>`.

3. Let `sndr` be an expression of type *`inline-sender`*, let `rcvr` be an expression such that `receiver_of<decltype((rcvr)), CS>` is `true` where `CS` is `completion_signatures<set_value_t()>`, then:

(3.1) — the expression `connect(sndr, rcvr)` has type *`inline-state`*`<remove_cvref_t<decltype((rcvr))>>` and is potentially-throwing if and only if `((void)sndr, auto(rcvr))` is potentially-throwing, and

~~(3.2)~~ — ~~the expression `get_completion_scheduler<set_value_t>(get_env(sndr))` has type `inline_scheduler` and is potentially-throwing if and only if `get_env(sndr)` is potentially-throwing.~~

[Editor's note: For reference: cplusplus/sender-receiver#349]

§ 5 Removing algorithm customization

This section discusses the implications of removing algorithm customizability.

Removing algorithm customization is fairly straightforward in most regards. The sender algorithm customization point objects would directly return the result of calling *`make-sender`* instead of passing it to `transform_sender`.

However, there are a few parts of `std::execution` that need special care when making the algorithms non-customizable.

§ 5.1 The parallel scheduler

The `parallel_scheduler` goes to great lengths to ensure that the `bulk` family of algorithms – `bulk`, `bulk_chunked`, and `bulk_unchunked` – are executed in parallel when the user requests it and when the underlying execution context supports it.

To that end, the `parallel_scheduler` "provides a customized implementation" of the `bulk_chunked` and `bulk_unchunked` algorithms, but nothing is said about how those custom implementations are found or under what circumstances users can be assured that the `parallel_scheduler` will use them. Arguably, this is under-specified in the current Working Draft and should be addressed whether this paper is accepted or not.

We have to give users a guarantee that if X, Y, and Z conditions are met, `bulk[_[un]chunked]` will be run in parallel with absolute certainty.

One solution is to say that the `bulk` algorithms are guaranteed to execute in parallel when the immediate predecessor of the `bulk` operation is known to complete on the `parallel_scheduler`. In a sender expression such as the following:

```
namespace ex = std::execution;
sndr | ex::bulk(ex::par, 1024, fn)
```

If `sndr`'s attributes advertizes a completion scheduler of type `parallel_scheduler`, then we can guarantee that the `bulk` operation will execute in parallel. Implementations can choose to parallelize `bulk` under other circumstances, but we require this one.

The implication of offering this guarantee is that we must preserve the guarantee going forward. Any new customization mechanism we might add must *never* result in parallel execution becoming serialized.

The reverse is not necessarily true though. I maintain that a future change that parallelizes a `bulk` algorithm that formerly executed serially on the `parallel_scheduler` is an acceptable change of behavior.

If SG1 or LEWG disagrees, there are ways to avoid even this behavior change.

§ 5.2 The task scheduler

Library issue #4336 describes the poor interaction between `task_scheduler`, a type-erased scheduler, and the `bulk` family of algorithms; namely, that the `task_scheduler` always executes `bulk` in serial, even when it is wrapping a `parallel_scheduler`.

This is not a problem caused by the customization mechanism, but it is something that can be addressed as part of the customization removal process.

When we address that issue, we must avoid the `parallel_scheduler` pitfall by under-specifying the interaction with `bulk`. As with `parallel_scheduler`, users must have a guarantee about the conditions under which `bulk` is accelerated on a `task_scheduler`.

Fortunately, the `parallel_scheduler` has already given us a way to punch the `bulk_chunked` and `bulk_unchunked` algorithms through a type-erased API boundary: `parallel_scheduler_backend` (33.16.4 [exec.sysctxrepl.psb]). By specifying the behavior of `task_scheduler` in terms of `parallel_scheduler_backend` and `bulk_item_receiver_proxy`, we can give `task_scheduler` the ability to parallelize `bulk` without having to invent a new mechanism.

§ 5.3 The `bulk` algorithms

Few users will ever have a need to customize an algorithm like `then` or `let_value`. The `bulk` algorithms are a different story. Anybody with a custom thread pool will benefit from a custom `bulk` implementation that can run in parallel on the thread pool. The loss of algorithm customization is particularly painful in this area. This section explores some options to address these concerns and makes a recommendation.

§ 5.3.1 Option 1: Remove `bulk`, `bulk_chunked`, and `bulk_unchunked`

This option cuts the Gordian knot, but comes at a high cost. The `parallel_scheduler` can hardly be called "parallel" if it does not offer a way to execute work in parallel, so cutting the `bulk` algorithms probably means cutting `parallel_scheduler` also.

§ 5.3.2 Option 2: Magical parallel execution

In this option, we keep the `bulk` algorithms and the `parallel_scheduler`, and we say that the `bulk` algorithms are executed in parallel on the `parallel_scheduler` (and on a `task_scheduler` that wraps a `parallel_scheduler`), but we leave the mechanism unspecified.

This option is essentially the *status quo*, except that as discussed in The parallel scheduler, this aspect of the `parallel_scheduler` is currently under-specified. The referenced section proposes a path forward.

A variant of this option is to specify an exposition-only mechanism whereby `bulk` gets parallelized.

This option makes `parallel_scheduler` and `task_scheduler` "magic" with respect to the `bulk` algorithms. End users would have no standard mechanism to parallelize `bulk` on their own third-party thread pools in C++26.

This is the approach taken by the Proposed wording, option 2: Remove algorithm customization below.

§ 5.3.3 Option 3: A normative mechanism for the `bulk`∗ algorithms only

In this option, we reintroduce algorithm customization with a special-purpose API just for the `bulk` algorithms. For example, a scheduler might have an optional `sch.bulk_transform(sndr, env)` that turns a serial `bulk`∗ sender into one that executes in parallel on

scheduler `sch`. Whenever a `bulk*` sender is passed to `connect`, `connect` can check the sender's predecessor for a completion scheduler that defines `bulk_transform` and uses it if found.

The downside of this approach is that we will still have to support this API even when a more general algorithm customization mechanism is available.

§ 5.4 Impacts on hardware vendors

Without algorithm customization, manufacturers of special-purpose hardware accelerators will not be able to ship a scheduler that both:

— works with any standard-conforming implementation of `std::execution`, and

— performs optimally on their hardware for all of the standard algorithms.

See Mitigating factors for some reasons why this is not as terrible as it could be.

§ 5.5 Mitigating factors

The loss of direct support for sender algorithm customization is a blow to power users of `std::execution`, but there are a few factors that mitigate the blow.

§ 5.5.1 Sender introspection

All of the senders returned from the standard algorithms are self-describing and can be unpacked into their constituent parts with structured bindings. A sufficiently motivated user can "customize" an algorithm by writing a recursive sender tree transformation, explicitly transforming senders before launching them.

§ 5.5.2 Third party algorithms

The sender concepts and customization points make it possible for users to write their own sender algorithms that interoperate with the standard ones. If a user wants to change the behavior of the `then` algorithm in some way, they have the option of writing their own and using it instead. I expect libraries of third-party algorithms to appear on GitHub in time, as they tend to.

§ 5.5.3 Difficulties with proprietary extensions

Some execution contexts place extra-standard requirements on the code that executes on them. For example, NVIDIA GPUs require device-accelerated code to be annotated with its proprietary `__device__` annotation. Standard libraries are unlikely to ship implementations of `std::execution` with such annotations. The consequence is that, rather than shipping just a GPU scheduler with some algorithm customizations, a vendor like NVIDIA is already committed to shipping its own complete implementation of `std::execution` (in a different namespace, of course).

For such vendors, the inability to customize standard algorithms is a moot point. Since it is *implementing* the standard algorithms, they can make the implementations do whatever they want.

§ 5.6 The removal process

§ 5.6.1 Approach

The approach to removing sender algorithm customization is twofold:

— Remove those components that facilitate algorithm customization and their uses where it is easy to do so.

— In all other cases, turn normative mechanisms into non-normative ones so we can change them later. This results in smaller and safer wording changes and preserves the already agreed-upon semantics in a way that is easy to verify.

§ 5.6.2 Procedure

The steps for removing algorithm customization are detailed below.

1. Remove the type `default_domain` (33.9.5 [exec.domain.default]).

2. Remove the functions:

   — `transform_sender` (33.9.6 [exec.snd.transform]),

   — `transform_env` (33.9.7 [exec.snd.transform.env]), and

   — `apply_sender` (33.9.8 [exec.snd.apply]).

3. Remove the query object `get_domain` (33.5.5 [exec.get.domain]).

4. Remove the exposition-only helpers:

   — *`completion-domain`* (33.9.2 [exec.snd.expos]/8-9),

   — *`get-domain-early`* (33.9.2 [exec.snd.expos]/13), and

   — *`get-domain-late`* (33.9.2 [exec.snd.expos]/14).

5. Change the functions `get_completion_signatures` (33.9.9 [exec.getcomplsigs]) and `connect` (33.9.10 [exec.connect]) to operate on a sender determined as follows instead of passing the sender through `transform_sender`:

   — If the sender has a tag with an exposition-only *`transform-sender`* member function, pass the sender to this function with the receiver's environment and then call `connect` on the resulting sender. This preserves the behavior of calling `transform_sender` with the `default_domain`.

   — Otherwise, call the `connect` member function of the passed-in sender.

6. For the following algorithms that are currently expressed in terms of a sender transformation to a lowered form, move the lowering from *alg*.`transform_sender(sndr, env)` to *alg.transform-sender*`(sndr, env)`.

   — `starts_on` (33.9.12.5 [exec.starts.on]),

   — `continues_on` (33.9.12.6 [exec.continues.on]),

   — `on` (33.9.12.8 [exec.on]),

   — `bulk` (33.9.12.11 [exec.bulk]),

   — `when_all_with_variant` (33.9.12.12 [exec.when.all]),

   — `stopped_as_optional` (33.9.12.14 [exec.stopped.opt]), and

   — `stopped_as_error` (33.9.12.15 [exec.stopped.err]).

7. For each sender adaptor algorithm in 33.9.12 [exec.adapt] that is specified to be expression-equivalent to some `transform_sender` invocation of the form:

```
transform_sender(some-computed-domain(),   make-sender(tag,
        {args...}, sndr));
```

   Change the expression to:

```
make-sender(tag, {args...}, sndr);
```

   For example, in 33.9.12.6 [exec.continues.on]/3, the following:

```
transform_sender(get-domain-early(sndr),   make-sender(con-
        tinues_on, sch, sndr))
```

   would be changed to:

```
make-sender(continues_on, sch, sndr)
```

   Additionally, if there is some caveat of the form "except that `sndr` is evaluated only once," that caveat should be removed as appropriate.

8. Merge the `schedule_from` (33.9.12.7 [exec.schedule.from]) and `continues_on` (33.9.12.6 [exec.continues.on]) algorithms into one algorithm called `continues_on`. (Currently they are separate so that they can be customized independently; by default `continues_on` merely dispatches to `schedule_from`.)

9. Change 33.9.13.1 [exec.sync.wait] and 33.9.13.2 [exec.sync.wait.var] to dispatch directly to their default implementations instead of computing a domain and using `apply_sender` to dispatch to an implementation.

10. Fix a bug in the `on(sndr, sch, closure)` algorithm where a `write_env` is incorrectly changing the "current" scheduler before its child `continues_on` actually transfers to that scheduler. `continues_on` needs to know the scheduler on which it will be started in order to find customizations correctly in the future.

11. Tweak the wording of `parallel_scheduler` (33.15 [exec.par.scheduler]) to indicate that it (`parallel_scheduler`) is permitted to run the `bulk` family of algorithms in parallel in accordance with those algorithms' semantics, rather than suggesting that those algorithms are "customized" for `parallel_scheduler`. The mechanism for such remains non-normative, however we specify the conditions under which the `parallel_scheduler` is guaranteed to run the `bulk` algorithms in parallel. (This is currently under-specified.)

12. Respecify `task_scheduler` in terms of `parallel_scheduler_backend` so that the `bulk` algorithms can be accelerated despite `task_scheduler`'s type-erasure. This addresses LWG#4336. As with `parallel_scheduler`, we specify the conditions under which `task_scheduler` is guaranteed to run the `bulk` algorithms in parallel.

13. From the `scheduler` concept, remove the required expression:

    ```
    { auto(get_completion_scheduler<set_value_t>(get_env(sched-
            ule(std::forward<Sch>(sch))))) }
       -> same_as<remove_cvref_t<Sch>>;
    ```

    Instead, add a semantic requirement that *if* the above expression is well-formed, then it shall compare equal to `sch`. Additionally, require that that expression is well-formed for the `parallel_scheduler`, the `task_scheduler`, and `run_loop`'s scheduler, but not `inline_scheduler`. See inline_scheduler improvements for the motivation behind these changes, but in short: the `inline_scheduler` will not know where it completes in C++26, but it will in C++29 when we add back algorithm customization.

14. *Optional, but recommended*: Change the `env<>::query` member function to accept optional additional arguments after the query tag. This restores the original design of `env` to that which was first proposed in [P3325R1] and which was approved by LEWG straw poll in St Louis. As described in Restoring algorithm customization in C++29, when asking a sender for its completion scheduler, the caller needs to pass extra information about where the operation will be started, and that will require `env<>::query` to accept extra arguments.

§ 5.7 Restoring algorithm customization in C++29

For C++29, we want the sender algorithms in `std::execution` to be customizable, with different implementations suited for different execution contexts. If we remove customization for C++26, how do we add it back without breaking code?

Recall that many senders do not know where they will complete until they know where they will be started, and that information is not currently provided when the sender is queried for its completion scheduler. This is the shoal on which algorithm customization has foundered, because without accurate information about where operations are executing, it is impossible to pick the right algorithm implementation.

Once the problem is stated plainly, the fix (or at least a major part of it) is obvious:

> When asking the sender where it will complete, *tell it where it will start*.

The implication of this is that so-called "early" customization, performed when constructing a sender, will not be coming back. The receiver's execution environment is not known when constructing a sender. C++29 will bring back "late" customization only.

§ 5.7.1 Completion scheduler enhancements

A paper targetting C++29 would propose that we extend the `get_completion_scheduler` query to support an optional environment argument. Given a sender `S` and receiver `R`, the query would look like:

```
// Pass the sender's attributes and the receiver's environment
        when computing
// the completion scheduler:
auto   sch   =   get_completion_scheduler<set_value_t>(get_env(S),
        get_env(R));
```

It will not be possible in C++26 to pass the receiver's environment in this way, making this a conforming extension since it would not change the meaning of any existing code.

This change will also make it possible to provide a completion scheduler for the error channel in more cases. That is often not possible today since many errors are reported inline on the context on which the operation is started. The receiver's environment knows where the operation will be started, so by passing it to the `get_completion_scheduler<set_error_t>` query, the error completion scheduler is knowable.

> **Note:**
> The paragraph above makes it sound like this would be changing the behavior for the `get_completion_scheduler<set_error_t>(get_env(sndr))` query. But that expression will behave as it always has. Only when called with the receiver's environment will any new behavior manifest; hence, this change is a pure extension.

By the way, this extension to `get_completion_scheduler` motivates the change to `env<>::query` described above in The removal process. Although we could decide to defer that change until it is needed in C++29, it seems best to me to make the change now.

§ 5.7.2 Domains

There are sender expressions that complete on an indeterminate scheduler based on runtime factors; `when_all` is a good example. This is the problem the `get_domain` query solved. So long as all of `when_all`'s child senders share a common domain tag – a property of the scheduler – we know the domain on which the `when_all` operation will complete, even though we do not know which scheduler it will complete on. The domain controls algorithm selection, not the scheduler directly.

So the plan will be to bring back a `get_domain` query in C++29. Additionally, just as it is necessary to have three `get_completion_scheduler` queries, one each for the three different completion channels, it is necessary to have three `get_completion_domain` queries for the times when the completion scheduler is indeterminate but the domain is known.

> **Note:**
> Above we say, "So long as all of `when_all`'s child senders share a common domain tag […]". This sounds like we are adding a new requirement to the `when_all` algorithm. However, this requirement will be met for all existing uses of `when_all`. Before C++29, all senders will be in the "default" domain, so they trivially all share a common domain.

Giving a non-default domain to a scheduler is the way to opt-in to algorithm customization. Prior to C++29, there would be no `get_*domain` queries, hence the addition of those queries in C++29 would not affect any existing schedulers. And the domain queries would be so-called "forwarding" queries, meaning they will automatically be passed through layers of sender adaptors. Users will not have to change their code in order for domain information to be propagated. As a result, this change is a pure extension.

§ 5.7.3 Customizing `connect`

Since C++29 would support only late (`connect`-time) customization, customizing an algorithm effectively amounts to customizing that algorithm's `connect` operation. By default, `connect(sndr, rcvr)` calls `sndr.connect(rcvr)`, but in C++29 there will be a way to do something different depending on the sender's attributes and the receiver's environment.

As described in Algorithm dispatching in connect, `connect` will compute two domains, the "starting" domain and the (value) "completion" domain:

Domain kind	Query
Starting domain	`get_domain(get_env(rcvr))`
Completion domain	`get_completion_domain<set_value_t>(get_env(sndr), get_env(rcvr))`

The `connect` CPO would use these two domains to optionally apply "starting" and "completing" transformations to the sender prior to calling `.connect(rcvr)` on it. This will be a conforming extension since no schedulers in C++26 will have a custom domain tag, and hence no sender transformations.

§ 5.7.4 The parallel and task schedulers and `bulk`

Once we have a general mechanism for customizing algorithms, we can consider changing `parallel_scheduler` and `task_scheduler` to use that mechanism to find parallel implementations of the `bulk` algorithms. In C++26, it is unspecified precisely how those sched-

ulers accelerate `bulk`, and we can certainly leave it that way for C++29. No change is often the safest change.

If we wanted to switch to using the new algorithm dispatch mechanics in C++29, I believe we can do so with minimal impact on existing code. Any behavior change would be an improvement, accelerating `bulk` operations that *should* have been accelerated but were not.

Consider the following sender:

```
starts_on(parallel_scheduler(), just() | bulk(fn))
```

In C++26, we can offer no iron-clad standard guarantee that this `bulk` operation will be accelerated even though it is executing on the parallel scheduler. The predecessor of `bulk`, `just()`, would not know where it will complete in C++26. There is no plumbing yet to tell it that it will be started on the parallel scheduler. As a result, it is QoI whether this `bulk` will execute in parallel or not.

But suppose we add a `get_completion_domain<set_value_t>` query to the `parallel_scheduler` such that the query returns an instance of a new type: `parallel_domain`. Now, when connecting the `bulk` sender, `connect` will ask for the predecessor's domain, passing also the receiver's environment. Now the `just()` sender is able to say where it completes: the domain where it starts, `get_domain(get_env(rcvr))`. This will return `parallel_domain{}`. `connect` would then use that information to find a parallel implementation of `bulk`.

As a result, in C++29 we could guarantee that this usage of `bulk` will be parallelized. For some stdlib implementations, this would be a behavior change: what once executed serially on a thread of the parallel scheduler now executes in parallel on many threads. Can that break working code? Yes, but only code that had already violated the preconditions of `bulk`: that `fn` can safely be called in parallel.

I do not believe this should be considered a breaking change, since any code that breaks is already broken.

All of the above is true also for `task_scheduler`, which merely adds an indirection to the call to `connect`. After the changes suggested by the "Remove" option, the `task_scheduler` accelerates `bulk` in the same way as `parallel_scheduler`.

> **Note:**
> If we assign `parallel_domain` to the `parallel_scheduler`, and we *also* add a requirement to `when_all` that all of its child operations share a common domain (see Domains), does that have the potential to break existing code? It would not. We would make `parallel_domain` inherit from `default_domain` so that `when_all` will compute the common domain as `default_domain` even if one child completes in the `parallel_domain`.

§ 5.8 Proposed wording, option 2: Remove algorithm customization

[Editor's note: In 33.4 [execution.syn], make the following changes:]

```
… as before …

namespace std::execution {
  // [exec.queries], queries
  struct get_domain_t { unspecified };
  struct get_scheduler_t { unspecified };
  struct get_delegation_scheduler_t { unspecified };
  struct get_forward_progress_guarantee_t { unspecified };
  template<class CPO>
    struct get_completion_scheduler_t { unspecified };
  struct get_await_completion_adaptor_t { unspecified };

  inline constexpr get_domain_t get_domain{};
  inline constexpr get_scheduler_t get_scheduler{};
           inline      constexpr      get_delegation_scheduler_t
         get_delegation_scheduler{};
  enum class forward_progress_guarantee;
         inline    constexpr    get_forward_progress_guarantee_t
         get_forward_progress_guarantee{};
  template<class CPO>
                    constexpr     get_completion_scheduler_t<CPO>
         get_completion_scheduler{};
          inline     constexpr     get_await_completion_adaptor_t
         get_await_completion_adaptor{};

… as before …

  // [exec.env], class template env
  template<queryable... Envs>
    struct env;

  // [exec.domain.default], execution domains
  struct default_domain;

  // [exec.sched], schedulers
  struct scheduler_t {};

… as before …

  template<sender Sndr>
    using tag_of_t = see below;

  // [exec.snd.transform], sender transformations
  template<class Domain, sender Sndr, queryable... Env>
      requires (sizeof...(Env) <= 1)
    constexpr sender decltype(auto) transform_sender(
       Domain dom, Sndr&& sndr, const Env&... env) noexcept(see
        below);

  // [exec.snd.transform.env], environment transformations
```

```
~~template<class Domain, sender Sndr, queryable Env>~~
~~  constexpr queryable decltype(auto) transform_env(~~
~~    Domain dom, Sndr&& sndr, Env&& env) noexcept;~~

~~// [exec.snd.apply], sender algorithm application~~
~~template<class Domain, class Tag, sender Sndr, class... Args>~~
~~  constexpr decltype(auto) apply_sender(~~
~~    Domain dom, Tag, Sndr&& sndr, Args&&... args) noexcept(see~~
~~      below);~~

// [exec.connect], the connect sender algorithm
struct connect_t;
inline constexpr connect_t connect{};

… as before …
```

[Editor's note: Remove subsection 33.5.5 [exec.get.domain].]

[Editor's note: In 33.6 [exec.sched], change paragraphs 1 and 5 and strike paragraph 6 as follows:]

1. The `scheduler` concept defines the requirements of a scheduler type (33.3 [exec.async.ops]). `schedule` is a customization point object that accepts a scheduler. A valid invocation of `schedule` is a schedule-expression.

```
namespace std::execution {
  template<class Sch>
    concept scheduler =
              derived_from<typename  remove_cvre-
        f_t<Sch>::scheduler_concept,      sched-
        uler_t> &&
      queryable<Sch> &&
      requires(Sch&& sch) {
         { schedule(std::forward<Sch>(sch)) } ->
        sender;
        ~~{ auto(get_completion_scheduler<set_val-~~
        ~~ue_t>(~~
              ~~get_env(schedule(std::forward<Sch>~~
        ~~(sch)))) }~~
            ~~-> same_as<remove_cvref_t<Sch>>;~~
      } &&
         equality_comparable<remove_cvref_t<Sch>>
        &&
      copyable<remove_cvref_t<Sch>>;
}
```

… as before …

5. For a given scheduler expression `sch`, if the expression `auto(get_completion_scheduler<set_value_t>(get_env(schedule(sch))))` is well-formed, it shall have type `remove_cvref_t<Sch>` and shall compare equal to `sch`.

6. ~~For a given scheduler expression `sch`, if the expression `get_domain(sch)` is well-formed, then the expression `get_domain(get_env(schedule(sch)))` is also well-formed and has the same type.~~

[Editor's note: In 33.9.1 [exec.snd.general], change paragraph 1 as follows:]

1. Subclauses 33.9.11 [exec.factories] and 33.9.12 [exec.adapt] define ~~customizable~~ algorithms that return senders. ~~Each algorithm has a default implementation.~~ Let `sndr` be the result of an invocation of such an algorithm or an object equal to the result (18.2 [concepts.equality]), and let `Sndr` be `decltype((sndr))`. Let `rcvr` be a receiver of type `Rcvr` with associated environment env of type `Env` such that `sender_to<Sndr, Rcvr>` is `true`. ~~For the default implementation of the algorithm that produced `sndr`, c~~Connecting `sndr` to `rcvr` and starting the resulting operation state (33.3 [exec.async.ops]) necessarily results in the potential evaluation (6.3 [basic.def.odr]) of a set of completion operations whose first argument is a subexpression equal to `rcvr`. Let `Sigs` be a pack of completion signatures corresponding to this set of completion operations, and let `CS` be the type of the expression `get_completion_signatures<Sndr, Env>()`. Then `CS` is a specialization of the class template `completion_signatures` (33.10 [exec.cmplsig]), the set of whose template arguments is `Sigs`. If none of the types in `Sigs` are dependent on the type `Env`, then the expression `get_completion_signatures<Sndr>()` is well-formed and its type is `CS`. ~~If a user-provided implementation of the algorithm that produced `sndr` is selected instead of the default:~~

(1.1) — ~~Any completion signature that is in the set of types denoted by `completion_signatures_of_t<Sndr, Env>` and that is not part of `Sigs` shall correspond to error or stopped completion operations, unless otherwise specified.~~

(1.2) — ~~If none of the types in `Sigs` are dependent on the type `Env`, then `completion_signatures_of_t<Sndr>` and `completion_signatures_of_t<Sndr, Env>` shall denote the same type.~~

[Editor's note: Change 33.9.2 [exec.snd.expos] paragraph 6 as follows:]

6. For a scheduler `sch`, *`SCHED-ATTRS`*`(sch)` is ~~an expression `o1` whose type satisfies *`queryable`* such that `o1.query(get_completion_scheduler<Tag>)` is an expression with the same type and value as `sch`~~ equivalent to *`MAKE-ENV`*`(get_completion_scheduler<set_value_t>, sch)` ~~where `Tag` is one of `set_value_t` or `set_stopped_t`, and such that `o1.query(get_domain)` is expression-equivalent to `sch.query(get_domain)`~~. *`SCHED-ENV`*`(sch)` is ~~an expression `o2` whose type satisfies *`queryable`* such that `o2.query(get_scheduler)` is a prvalue with the same type and value~~

~~as sch, and such that o2.query(get_domain) is expression-equivalent to sch.query(get_domain)~~ equivalent to *MAKE-ENV*(get_scheduler, sch).

[Editor's note: Remove the prototype of the exposition-only *completion-domain* function just before 33.9.2 [exec.snd.expos] paragraph 8, and with it remove paragraphs 8 and 9, which specify the function's behavior.]

[Editor's note: Remove 33.9.2 [exec.snd.expos] paragraphs 13 and 14 and the prototypes for the *get-domain-early* and *get-domain-late* functions.]

[Editor's note: Remove subsection 33.9.5 [exec.domain.default].]

[Editor's note: Remove subsection 33.9.6 [exec.snd.transform].]

[Editor's note: Remove subsection 33.9.7 [exec.snd.transform.env].]

[Editor's note: Remove subsection 33.9.8 [exec.snd.apply].]

[Editor's note: Change 33.9.9 [exec.getcomplsigs] as follows:]

1. Let *except* be an rvalue subexpression of an unspecified class type *Except* such that move_constructible<*Except*> && derived_from<*Except*, exception> is true. Let *CHECKED-COMPLSIGS*(*e*) be *e* if *e* is a core constant expression whose type satisfies *valid-completion-signatures*; otherwise, it is the following expression:

```
(e, throw except, completion_signatures())
```

   Let *get-complsigs*<Sndr, Env...>() be expression-equivalent to remove_reference_t<Sndr>::template get_completion_signatures<Sndr, Env...>(). ~~Let NewSndr be Sndr if sizeof...(Env) == 0 is true; otherwise, decltype(*s*) where *s* is the following expression:~~ Let NewSndr be decltype(tag_of_t<Sndr>().*transform-sender*(declval<Sndr>(), declval<Env>()...)) if that expression is well-formed, and Sndr otherwise.

```
~~transform_sender(~
~  get-domain-late(declval<Sndr>(), declval<Env>()...),~
~  declval<Sndr>(),~
~  declval<Env>()...)~
```

2. *Constraints*: sizeof...(Env) <= 1 is true.

3. *Effects*: Equivalent to: *… as before …*

[Editor's note: Change 33.9.10 [exec.connect] as follows:]

1. `connect` connects (33.3 [exec.async.ops]) a sender with a receiver.

2. The name `connect` denotes a customization point object. For subexpressions `sndr` and `rcvr`, let `Sndr` be `decltype((sndr))` and `Rcvr` be `decltype((rcvr))`~~.~~; let `new_sndr` be the expression ~~`transform_sender(decltype(get_domain_late(sndr, get_env(rcvr))){}, sndr, get_env(rcvr))`~~ `tag_of_t<Sndr>().transform_sender(sndr, get_env(rcvr))` if that expression is well-formed, and `sndr` otherwise; and let `DS` and `DR` be `decay_t<decltype((new_sndr))>` and `decay_t<Rcvr>`, respectively.

3. Let *`connect-awaitable-promise`* be *… as before …*

[Editor's note: Change 33.9.11.1 [exec.schedule] paragraph 4 as follows:]

4. If the expression

```
get_completion_scheduler<set_value_t>(get_env(sch.sched-
        ule()))== sch
```

is ~~ill-formed or~~ well-formed and does not evaluate~~s~~ to ~~`false`~~ `sch`, the behavior of calling `schedule(sch)` is undefined.

[Editor's note: From 33.9.12.1 [exec.adapt.general], strike paragraph (3.6) as follows:]

3. Unless otherwise specified:

… as before …

(3.5) — An adaptor whose child senders are all non-dependent (33.3 [exec.async.ops]) is itself non-dependent.

(3.6) — ~~These requirements apply to any function that is selected by the implementation of the sender adaptor.~~

(3.7) — *Recommended practice*: Implementations should use the completion signatures of the adaptors to communicate type errors to users and to propagate any such type errors from child senders.

[Editor's note: Change 33.9.12.5 [exec.starts.on] paragraph 3 as follows:]

3. Otherwise, the expression `starts_on(sch, sndr)` is expression-equivalent to~~:~~ `make-sender(starts_on, sch, sndr)`.

```
transform_sender(
          query_with_default(get_domain,      sch,
        default_domain()),
  make_sender(starts_on, sch, sndr))
```

~~except that `sch` is evaluated only once.~~

4. Let `out_sndr` and `env` be subexpressions such that `OutSndr` is `decltype((out_sndr))`. If *sender-for*<`OutSndr`, `starts_on_t`> is `false`, then the ~~expressions `starts_on.transform_env(out_sndr, env)` and~~ expression `starts_on.`~~`transform_sender`~~*`transform-sender`*`(out_sndr, env)` ~~are~~ is ill-formed; otherwise it is equivalent to:

(4.1) — ~~`starts_on.transform_env(out_sndr, env)` is equivalent to:~~

```
auto&& [_, sch, _] = out_sndr;
return JOIN-ENV(SCHED-ENV(sch), FWD-ENV(env));
```

(4.2) — ~~`starts_on.transform_sender(out_sndr, env)` is equivalent to:~~

```
auto&& [_, sch, sndr] = out_sndr;
return let_value(
  schedule(sch),
  [sndr = std::forward_like<OutSndr>(sndr)]
        () mutable
              noexcept(is_nothrow_move_con-
        structible_v<decay_t<OutSndr>>) {
    return std::move(sndr);
  });
```

5. Let `out_sndr` be *… as before …*

[Editor's note: Remove subsection 33.9.12.6 [exec.continues.on]]

[Editor's note: Change 33.9.12.7 [exec.schedule.from] to [exec.continues.on] and change it as follows:]

33.9.12.~~7~~6 `execution::`~~`schedule_from`~~`continues_on` [exec.~~sched-ule.from~~.continues.on]

1. ~~`schedule_from`~~`continues_on` schedules work dependent on the completion of a sender onto a scheduler's associated execution resource.

   ~~[*Note 1*: `schedule_from` is not meant to be used in user code; it is used in the implementation of `continues_on`. — end note]~~

2. The name ~~`schedule_from`~~`continues_on` denotes a customization point object. For some subexpressions `sch` and `sndr`, let `Sch` be `decltype((sch))` and `Sndr` be `decltype((sndr))`. If `Sch` does not satisfy scheduler, or `Sndr` does not satisfy `sender`, ~~`schedule_from(sch, sndr)`~~`continues_on(sndr, sch)` is ill-formed.

3. Otherwise, the expression ~~`schedule_from(sch, sndr)`~~`continues_on(sndr, sch)` is expression-equivalent to~~:~~ *`make-sender`*`(continues_on, sch, sndr)`

```
transform_sender(
          query-with-default(get_domain,     sch,
        default_domain()),
   make-sender(schedule_from, sch, sndr))
```

except that sch is evaluated only once.

4. The exposition-only class template *impls-for* (33.9.1 [exec.snd.general]) is specialized for schedule_from_tcontinues_on_t as follows:

```
namespace std::execution {
   template<>
        struct   impls-for<schedule_from_tcontin-
        ues_on_t> : default-impls {
         static constexpr auto get-attrs = see
        below;
         static constexpr auto get-state = see
        below;
          static constexpr auto complete = see
        below;

      template<class Sndr, class... Env>
         static consteval void check-types();
   };
}
```

5. The member *impls-for*<schedule_from_tcontinues_on_t>::*get-attrs* is initialized with a callable object equivalent to the following lambda:

```
[](const auto& data, const auto& child) noex-
        cept -> decltype(auto) {
       return   JOIN-ENV(SCHED-ATTRS(data),   FWD-
        ENV(get_env(child)));
}
```

6. The member *impls-for*<schedule_from_tcontinues_on_t>::*get-state* is initialized with a callable object equivalent to the following lambda:

…as before …

```
template<class Sndr, class... Env>
  static consteval void check-types();
```

…as before …

12. The member *impls-for*<schedule_from_tcontinues_on_t>::*complete* is initialized with a callable object equivalent to the following lambda:

…as before …

13. Let `out_sndr` be a subexpression denoting a sender returned from ~~`schedule_from(sch, sndr)`~~`continues_on(sndr, sch)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be *… as before …*

[Editor's note: Change 33.9.12.8 [exec.on] paragraphs 3-8 as follows:]

3. Otherwise, if `decltype((sndr))` satisfies `sender`, the expression `on(sch, sndr)` is expression-equivalent to~~:~~ *`make-sender`*`(on, sch, sndr)`.

```
~~transform_sender(~~
~~        query-with-default(get_domain(sndr), sch,~~
~~    default_domain()),~~
~~  make-sender(on, sch, sndr))~~
```

~~except that `sch` is evaluated only once.~~

4. For subexpressions `sndr`, `sch`, and `closure`, if

(4.1) — `decltype((sch))` does not satisfy `scheduler`, or

(4.2) — `decltype((sndr))` does not satisfy `sender`, or

(4.3) — `closure` is not a pipeable sender adaptor closure object ([exec.adapt.obj]), the expression `on(sndr, sch, closure)` is ill-formed; otherwise, it is expression-equivalent to~~:~~ *`make-sender`*`(on, `*`product-type`*`{sch, closure}, sndr)`.

```
~~transform_sender(~~
~~  get-domain-early(sndr),~~
~~      make-sender(on, product-type{sch,~~
~~    closure}, sndr))~~
```

~~except that `sndr` is evaluated only once.~~

5. Let `out_sndr` and `env` be subexpressions, let `OutSndr` be `decltype((out_sndr))`, and let `Env` be `decltype((env))`. If *`sender-for`*`<OutSndr, on_t>` is `false`, then the ~~expressions `on.transform_env(out_sndr, env)` and~~ expression `on.`~~`transform_sender`~~*`transform-sender`*`(out_sndr, env)` ~~are~~ is ill-formed.

6. Otherwise: Let *`not-a-scheduler`* be an unspecified empty class type.

7. ~~The expression `on.transform_env(out_sndr, env)` has effects equivalent to:~~

```
~~auto&& [_, data, _] = out_sndr;~~
~~if constexpr (scheduler<decltype(data)>) {~~
~~                  return          JOIN-ENV(SCHED-~~
~~      ENV(std::forward_like<OutSndr>(data)),~~
~~      FWD-ENV(std::forward<Env>(env)));~~
```

```
} else {
  return std::forward<Env>(env);
}
```

8. The expression `on.`~~`transform_sender`~~*`transform-sender`*`(out_sndr, env)` has effects equivalent to:

```
auto&& [_, data, child] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
   auto orig_sch =
          query-with-default(get_scheduler,  env,
        not-a-scheduler());

      if  constexpr  (same_as<decltype(orig_sch),
        not-a-scheduler>) {
      return not-a-sender{};
   } else {
      return continues_on(
            starts_on(std::forward_like<OutSndr>
        (data),       std::forward_like<OutSndr>
        (child)),
         std::move(orig_sch));
   }
} else {
   auto& [sch, closure] = data;
   auto orig_sch = query-with-default(
      get_completion_scheduler<set_value_t>,
      get_env(child),
          query-with-default(get_scheduler,  env,
        not-a-scheduler()));

      if  constexpr  (same_as<decltype(orig_sch),
        not-a-scheduler>) {
      return not-a-sender{};
   } else {
      return write_env continues_on(
         continues_on write_env(
            std::forward_like<OutSndr>(closure)(
               continues_on(
                        write_env(std::forward_-
        like<OutSndr>(child),             SCHED-
        ENV(orig_sch)),
                  sch)),
            orig_sch SCHED-ENV(sch)),
         SCHED-ENV(sch) orig_sch);
   }
}
```

[Editor's note: Change 33.9.12.9 [exec.then] paragraph 3 as follows:]

3. Otherwise, the expression *`then-cpo`*`(sndr, f)` is expression-equivalent to~~:~~ *`make-sender(then-cpo, f, sndr)`*.

```
transform_sender(get-domain-early(sndr),    make-
        sender(then-cpo, f, sndr))
```

~~except that `sndr` is evaluated only once.~~

[Editor's note: Change 33.9.12.10 [exec.let] paragraphs 2-4 as follows:]

2. For `let_value`, `let_error`, and `let_stopped`, let *`set-cpo`* be `set_value`, `set_error`, and `set_stopped`, respectively. Let the expression *`let-cpo`* be one of `let_value`, `let_error`, or `let_stopped`. For a subexpression `sndr`, let *`let-env`*`(sndr)` be expression-equivalent to the first well-formed expression below:

(2.1) — *`SCHED-ENV`*`(`*`get_completion_scheduler`*`<`*`decayed-typeof`*`<`*`set-cpo`*`>>(get_env(sndr)))`

~~(2.2) — *`MAKE-ENV(get_domain, get_domain(get_env(sndr)))`*~~

(2.3) — `(void(sndr), env<>{})`

3. The names `let_value`, `let_error`, and `let_stopped` denote *… as before …*

4. Otherwise, the expression *`let-cpo`*`(sndr, f)` is expression-equivalent to~~:~~ *`make-sender(let-cpo, f, sndr).`*

~~`transform_sender(`*`get-domain-early`*`(sndr), `*`make-sender(let-cpo, f, sndr))`~~

~~except that `sndr` is evaluated only once.~~

[Editor's note: Change 33.9.12.11 [exec.bulk] paragraphs 3 and 4 and insert paragraphs 5 and 6 as follows:]

3. Otherwise, the expression *`bulk-algo`*`(sndr, policy, shape, f)` is expression-equivalent to:

```
~~transform_sender(get-domain-early(sndr),~~ make-sender(
   bulk-algo, product-type<see below, Shape, Func>{policy,
        shape, f}, sndr)~~)~~
```

~~except that `sndr` is evaluated only once.~~ The first template argument of *`product-type`* is `Policy` if `Policy` models `copy_constructible`, and `const Policy&` otherwise.

4. Let `sndr` ~~and `env` be subexpressions~~ be an expression such that `Sndr` is `decltype((sndr))`. If *`sender-for`*`<Sndr, bulk_t>` is `false`, then the expression ~~`bulk.transform_sender(sndr, env)`~~ *`as-bulk-chunked(sndr)`* is ill-formed; otherwise, it is equivalent to:

```
auto [_, data, child] = sndr;
auto& [policy, shape, f] = data;
auto new_f = [func = std::move(f)](Shape begin,
        Shape end, auto&&... vs)
    noexcept(noexcept(f(begin, vs...))) {
```

```
  while (begin != end)
    func(begin++, vs...);
}
return   bulk_chunked(std::move(child),   policy,
        shape, std::move(new_f));
```

~~[*Note:* This causes the `bulk(sndr, policy, shape, f)` sender to be expressed in terms of `bulk_chunked(sndr, policy, shape, f)` when it is connected to a receiver whose execution domain does not customize `bulk`. — *end note*]~~

5. Let `sndr` and `env` be subexpressions, let `Sndr` be `decltype((sndr))`, and let `sch` be expression-equivalent to `get_completion_scheduler<set_value_t>(get_env(sndr.get<2>()))`. If *`sender-for`*`<Sndr, `*`decayed-typeof<bulk-algo>>` is `false`, the expression *`bulk-algo.transform-sender`*`(sndr, env)` is ill-formed; otherwise, it is expression-equivalent to:

(6.1) — `sch.`*`bulk-transform`*`(sndr, env)` if that expression is well-formed; otherwise,

(6.2) — `sch.`*`bulk-transform`*`(`*`as-bulk-chunked`*`(sndr), env)` if that expression is well-formed; otherwise,

(6.3) — *`bulk-algo.transform-sender`*`(sndr, env)` is ill-formed.

[Editor's note: Change 33.9.12.12 [exec.when.all] as follows:]

1. `when_all` and `when_all_with_variant` both *… as before …*

2. The names `when_all` and `when_all_with_variant` denote customization point objects. Let `sndrs` be a pack of subexpressions~~,~~ and let `Sndrs` be a pack of the types `decltype((sndrs))...`~~, and let `CD` be the type `common_type_t<decltype(`*`get-domain-early`*`(sndrs))...>`. Let `CD2` be `CD` if `CD` is well-formed, and `default_domain` otherwise~~. The expressions `when_all(sndrs...)` and `when_all_with_variant(sndrs...)` are ill-formed if any of the following is `true`:

(2.1) — `sizeof...(sndrs)` is `0`, or

(2.2) — `(sender<Sndrs> && ...)` is `false`.

3. The expression `when_all(sndrs...)` is expression-equivalent to~~:~~ *`make-sender`*`(when_all, {}, sndrs...)`.

```
transform_sender(CD2(),     make_sender(when_all,
        {}, sndrs...))
```

4. The exposition-only class template *`impls-for`* (33.9.1 [exec.snd.general]) is specialized for `when_all_t` as follows:

```
namespace std::execution {
   template<>
    struct impls-for<when_all_t> : default-impls
        {
         static constexpr auto get-attrs = see
        below;
      static constexpr auto get-env = see below;
         static constexpr auto get-state = see
        below;
      static constexpr auto start = see below;
         static constexpr auto complete = see
        below;

      template<class Sndr, class... Env>
         static consteval void check-types();
   };
}
```

… as before …

9. *Throws*: Any exception thrown as a result of evaluating the *Effects*~~, or an exception of an unspecified type derived from `exception` when `CD` is ill-formed~~.

10. ~~The member *`impls-for`*`<when_all_t>::get-attrs` is initialized with a callable object equivalent to the following lambda expression:~~

```
[](auto&&, auto&&... child) noexcept {
   if constexpr (same_as<CD, default_domain>) {
      return env<>();
   } else {
      return MAKE-ENV(get_domain, CD());
   }
}
```

… as before …

19. The expression `when_all_with_variant(sndrs...)` is expression-equivalent to~~:~~ *`make-sender`*`(when_all_with_variant, {}, sndrs...)`.

```
transform_sender(CD2(), make-sender(when_al-
        l_with_variant, {}, sndrs...));
```

20. Given subexpressions `sndr` and `env`, if *`sender-for`*`<decltype((sndr)), when_all_with_variant_t>` is `false`, then the expression `when_all_with_variant.`~~`transform_sender`~~*`transform-sender`*`(sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, _, ...child] = sndr;
return       when_all(into_variant(std::forward_-
        like<decltype((sndr))>(child))...);
```

[*Note 1*: This causes the `when_all_with_variant(sndrs...)` sender to become `when_all(into_variant(sndrs)...)` when it is connected with a receiver ~~whose execution domain does not customize `when_all_with_variant`~~. — *end note*]

[Editor's note: Change 33.9.12.13 [exec.into.variant] paragraph 3 as follows:]

3. Otherwise, the expression `into_variant(sndr)` is expression-equivalent to~~:~~ *`make-sender`*`(into_variant, {}, sndr)`.

~~`transform_sender(get-domain-early(sndr), make-sender(into_variant, {}, sndr))`~~

~~except that `sndr` is only evaluated once.~~

[Editor's note: Change 33.9.12.14 [exec.stopped.opt] paragraphs 2 and 4 as follows:]

2. The name `stopped_as_optional` denotes a pipeable sender adaptor object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `stopped_as_optional(sndr)` is expression-equivalent to~~:~~ *`make-sender`*`(stopped_as_optional, {}, sndr)`.

~~`transform_sender(get-domain-early(sndr), make-sender(stopped_as_optional, {}, sndr))`~~

~~except that `sndr` is only evaluated once.~~

3. The exposition-only class template *`impls-for`* *… as before …*

4. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If *`sender-for`*`<Sndr, stopped_as_optional_t>` is `false` then the expression `stopped_as_optional.`~~`transform_sender`~~*`transform-sender`*`(sndr, env)` is ill-formed; otherwise, if `sender_in<`*`child-type`*`<Sndr>, `*`FWD-ENV-T`*`(Env)>` is `false`, the expression `stopped_as_optional.`~~`transform_sender`~~*`transform-sender`*`(sndr, env)` is equivalent to *`not-a-sender`*`()`; otherwise, it is equivalent to:

```
auto&& [_, _, child] = sndr;
using    V    =    single-sender-value-type<child-
         type<Sndr>, FWD-ENV-T(Env)>;
return let_stopped(
  then(std::forward_like<Sndr>(child),
              []<class... Ts>(Ts&&... ts) noex-
         cept(is_nothrow_constructible_v<V,
         Ts...>) {
                    return optional<V>(in_place,
         std::forward<Ts>(ts)...);
         }),
    []() noexcept { return just(optional<V>());
         });
```

[Editor's note: Change 33.9.12.15 [exec.stopped.err] paragraphs 2 and 3 as follows:]

2. The name `stopped_as_error` denotes a pipeable sender adaptor object. For some subexpressions `sndr` and `err`, let `Sndr` be `decltype((sndr))` and let `Err` be `decltype((err))`. If the type `Sndr` does not satisfy `sender` or if the type `Err` does not satisfy *`movable-value`*, `stopped_as_error(sndr, err)` is ill-formed. Otherwise, the expression `stopped_as_error(sndr)` is expression-equivalent to~~:~~ *`make-sender(stopped_as_error, err, sndr)`*.

~~`transform_sender(get-domain-early(sndr), make-sender(stopped_as_error, err, sndr))`~~

~~except that `sndr` is only evaluated once.~~

3. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If *`sender-for`*`<Sndr, stopped_as_error_t>` is `false` then the expression `stopped_as_error.`~~`transform_sender`~~*`transform-sender`*`(sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, err, child] = sndr;
using E = decltype(auto(err));
return let_stopped(
  std::forward_like<Sndr>(child),
   [err = std::forward_like<Sndr>(err)]() noex-
        cept(is_nothrow_move_con-
        structible_v<E>) {
    return just_error(std::move(err));
  });
```

[Editor's note: Change 33.9.12.16 [exec.associate] paragraph 10 as follows:]

10. The name `associate` denotes a pipeable sender adaptor object. For subexpressions `sndr` and `token`:

(10.1) — If `decltype((sndr))` does not satisfy `sender`, or `remove_cvref_t<decltype((token))>` does not satisfy `scope_token`, then `associate(sndr, token)` is ill-formed.

(10.2) — Otherwise, the expression `associate(sndr, token)` is expression-equivalent to~~:~~ *`make-sender(associate, associate-data(token, sndr))`*.

~~`transform_sender(get-domain-early(sndr), make-sender(associate, associate-data(token, sndr)))`~~

~~except that `sndr` is evaluated only once.~~

[Editor's note: Change 33.9.13.1 [exec.sync.wait] paragraphs 4 and 9 as follows:]

4. The name `this_thread::sync_wait` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `this_thread::sync_wait(sndr)` is expression-equivalent to ~~the following, except that `sndr` is evaluated only once:~~ `sync_wait.apply(sndr)`, where *`apply`* is the exposition-only member function specified below.

~~`apply_sender(get-domain-early(sndr), sync_wait, sndr)`~~

Mandates:

(4.1) — `sender_in<Sndr, sync-wait-env>` is true.

(4.2) — The type *`sync-wait-result-type`*`<Sndr>` is well-formed.

~~(4.3)~~ — ~~`same_as<decltype(e), sync-wait-result-type<Sndr>>` is `true`, where `e` is the `apply_sender` expression i~~>

… as before …

9. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `sender_to<Sndr, sync-wait-receiver<Sndr>>` is `false`, the expression `sync_wait.`~~`apply_sender`~~*`apply`*`(sndr)` is ill-formed; otherwise, it is equivalent to:

```
sync-wait-state<Sndr> state;
auto op = connect(sndr, sync-wait-receiver<Sndr>{&state});
start(op);

state.loop.run();
if (state.error) {
  rethrow_exception(std::move(state.error));
}
return std::move(state.result);
```

[Editor's note: Change *Note 1* in 33.9.13.1 [exec.sync.wait] paragraph 10.1 as follows:]

[*Note 1*: The ~~default~~ implementation of `sync_wait` achieves forward progress guarantee delegation by providing a `run_loop` scheduler via the `get_delegation_scheduler` query on the *`sync-wait-receiver`*'s environment. The `run_loop` is driven by the current thread of execution. — *end note*]

[Editor's note: Change 33.9.13.2 [exec.sync.wait.var] paragraphs 1 and 2 as follows:]

1. The name `this_thread::sync_wait_with_variant` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype(into_variant(sndr))`. The expression `this_thread::sync_wait_with_variant(sndr)` is expression-equivalent to ~~the following, except `sndr` is evaluated only once:~~ `sync_wait_with_vari-`

ant.*apply*(sndr), where *apply* is the exposition-only member function specified below.

~~apply_sender(get_domain_early(sndr), sync_wait_with_variant, sndr)~~

Mandates:

(1.1) — `sender_in<Sndr, sync-wait-env>` is `true`.

(1.2) — The type *sync-wait-with-variant-result-type*<Sndr> is well-formed.

~~(1.3) — same_as<decltype(e), sync-wait-with-variant-result-type<Sndr>> is true, where e is the apply_sender expression i>~~

2. The expression sync_wait_with_variant.~~apply_sender~~*apply*(sndr) is equivalent to:

```
using result_type = sync-wait-with-variant-result-type<Sndr>;
if (auto opt_value = sync_wait(into_variant(sndr))) {
  return result_type(std::move(get<0>(*opt_value)));
}
return result_type(nullopt);
```

[Editor's note: Change *Note 1* in 33.9.13.1 [exec.sync.wait] paragraph 10.1 as follows:]

[*Note 1*: The ~~default~~ implementation of sync_wait_with_variant achieves forward progress guarantee delegation (6.10.2.3 [intro.progress]) by relying on the forward progress guarantee delegation provided by sync_wait. — *end note*]

[Editor's note: Change 33.11.2 [exec.env] as follows:]

```
namespace std::execution {
  template<queryable... Envs>
  struct env {
    Envs_0 envs_0;                // exposition only
    Envs_1 envs_1;                // exposition only
      ⋮
    Envs_n-1 envs_n-1;            // exposition only

    template<class QueryTag, class... Args>
      constexpr decltype(auto) query(QueryTag q,
        Args&&... args) const noexcept(see below);
  };

  template<class... Envs>
    env(Envs...) -> env<unwrap_reference_t<Envs>...>;
}
```

1. The class template `env` is used to construct a queryable object from several queryable objects. Query invocations on the resulting object are resolved by attempting to query each subobject in lexical order.

   … as before …

```
template<class QueryTag, class... Args>
constexpr decltype(auto) query(QueryTag q, Args&&...
        args) const noexcept(see below);
```

5. Let *has-query* be the following exposition-only concept:

```
template<class Env, class QueryTag, class...
        Args>
  concept has-query =                    // expo-
        sition only
    requires (const Env& env, Args&&... args) {
        env.query(QueryTag(), std::forward<Args>
        (args)...);
    };
```

6. Let *fe* be the first element of $envs_0$, $envs_1$, … $envs_{n-1}$ such that the expression `fe.query(q, std::forward<Args>(args)...)` is well-formed.

7. *Constraints*: `(has-query<Envs, QueryTag, Args...> || ...)` is `true`.

8. *Effects*: Equivalent to: `return fe.query(q, std::forward<Args>(args)...);`

9. *Remarks*: The expression in the `noexcept` clause is equivalent to `noexcept(fe.query(q, std::forward<Args>(args)...))`.

[Editor's note: In 33.12.1.2 [exec.run.loop.types], add a new paragraph after paragraph 4 as follows:]

4. Let *sch* be an expression of type *run-loop-scheduler*. The expression `schedule(sch)` has type *run-loop-sender* and is not potentially-throwing if *sch* is not potentially-throwing.

5. For type *set-tag* other than `set_error_t`, the expression `get_completion_scheduler<set-tag>(get_env(schedule(sch))) == sch` evaluates to `true`.

[Editor's note: Change 33.13.3 [exec.affine.on] paragraph 3 as follows:]

3. Otherwise, the expression `affine_on(sndr, sch)` is expression-equivalent to~~:~~ *make-sender*`(affine_on, sch, sndr)`.

~~transform_sender(get_domain_early(sndr), make_sender(affine_on, sch, sndr))~~

~~except that sndr is evaluated only once.~~

[Editor's note: Change paragraph 3 of 33.13.4 [exec.inline.scheduler] as follows:]

3. Let sndr be an expression of type *inline-sender*, let rcvr be an expression such that receiver_of<decltype((rcvr)), CS> is true where CS is completion_signatures<set_value_t()>, then~~:~~ [Editor's note: Move the text of (3.1) below into this paragraph.]

(3.1) the expression connect(sndr, rcvr) has type *inline-state*<remove_cvref_t<decltype((rcvr))>> and is potentially-throwing if and only if ((void)sndr, auto(rcvr)) is potentially-throwing~~, and~~.

~~(3.2) the expression get_completion_scheduler<set_value_t>(get_env(sndr)) has type inline_scheduler and is potentially-throwing if and only if get_env(sndr) is potentially-throwing.~~

[Editor's note: Change 33.13.5 [exec.task.scheduler] as follows:]

```
namespace std::execution {
  class task_scheduler {
    class ts-sender;                          // exposition only

    template<receiver R>
      class state;                            // exposition only

    template<class Sch>
      class backend-for;                      // exposition only
  public:
    using scheduler_concept = scheduler_t;

    template<class Sch, class Allocator = allocator<void>>
        requires (!same_as<task_scheduler, remove_cvref_t<Sch>>)
         && scheduler<Sch>
    explicit task_scheduler(Sch&& sch, Allocator alloc = {});

    ts-sendersee below schedule();

    template<class Sndr, class Env>       // exposition only
      see below bulk-transform(Sndr&& sndr, const Env& env);

      friend bool operator==(const task_scheduler& lhs, const
        task_scheduler& rhs) noexcept;

    template<class Sch>
      requires (!same_as<task_scheduler, Sch>) && scheduler<Sch>
      friend bool operator==(const task_scheduler& lhs, const
        Sch& rhs) noexcept;

  private:
```

```
    shared_ptr<void parallel_scheduler_backend> sch_; // exposi-
       tion only
                                                          // see
       [exec.sysctxrepl.psb]
  };
}
```

1. `task_scheduler` is a class that models `scheduler` (33.6 [exec.sched]). Given an object `s` of type `task_scheduler`, let *`SCHED`*`(s)` be the *`sched_`* member of the object owned by `s.`*`sch_`*.

2. For an lvalue `r` of type derived from `receiver_proxy`, let *`WRAP-RCVR`*`(r)` be an object of a type that models `receiver` and whose completion handlers result in invoking the corresponding completion handlers of `r`.

```
template<class Sch>
struct backend-for : parallel_scheduler_backend {
        // exposition only
          explicit backend-for(Sch sch) :
        sched_(std::move(sch)) {}

    void schedule(receiver_proxy& r, span<byte> s)
        noexcept override;
        void schedule_bulk_chunked(size_t shape,
        bulk_item_receiver_proxy& r,
                                span<byte> s) noexcept
        override;
       void schedule_bulk_unchunked(size_t shape,
        bulk_item_receiver_proxy& r,
                                span<byte> s) noexcept
        override;

  Sch sched_; // exposition only
};
```

3. Let `sndr` be a sender whose only value completion signature is `set_value_t()` and for which the expression `get_completion_scheduler<set_value_t>(get_env(sndr)) ==` *`sched_`* is `true`.

```
void schedule(receiver_proxy& r, span<byte> s) noex-
        cept override;
```

4. *Effects*: Constructs an operation state `os` with `connect(schedule(`*`sched_`*`), `*`WRAP-RCVR`*`(r))` and calls `start(os)`.

```
void schedule_bulk_chunked(size_t shape,
        bulk_item_receiver_proxy& r,
                                span<byte> s) noexcept
        override;
```

5. *Effects*: Let `chunk_size` be an integer less than or equal to `shape`, let `num_chunks` be `(shape + chunk_size - 1) / chunk_size`, and let `fn`

be a function object such that for an integer `i`, `fn(i)` calls `r.execute(i * chunk_size, m)`, where `m` is the lesser of `(i + 1) * chunk_size` and `shape`. Constructs an operation state `os` as if with `connect(bulk(sndr, par, num_chunks, fn), WRAP-RCVR(r))` and calls `start(os)`.

```
void       schedule_bulk_unchunked(size_t       shape,
        bulk_item_receiver_proxy& r,
                                   span<byte> s) noexcept
        override;
```

6. *Effects*: Let `fn` be a function object such that for an integer `i`, `fn(i)` is equivalent to `r.execute(i, i + 1)`. Constructs an operation state `os` as if with `connect(bulk(sndr, par, shape, fn), WRAP-RCVR(r))` and calls `start(os)`.

```
template<class      Sch,      class      Allocator      =
        allocator<void>>
      requires(!same_as<task_scheduler,   remove_cvre-
        f_t<Sch>>) && scheduler<Sch>
explicit  task_scheduler(Sch&& sch, Allocator alloc =
        {});
```

2. *Effects*: Initialize *sch_* with `allocate_shared<`*backend-for*`<re-move_cvref_t<Sch>>>(alloc, std::forward<Sch>(sch))`.

[Editor's note: Paragraphs 3-7 are kept unmodified. Remove paragraphs 8-12 and add the following paragraphs:]

```
see below schedule();
```

8. *Returns*: a prvalue `sndr` whose type `Sndr` models `sender` such that:

(8.1) — `get_completion_scheduler<set_value_t>(get_env(sndr))` is equal to `*this`.

(8.2) — If a receiver `rcvr` is connected to `sndr` and the resulting operation state is started, calls *`sch_`*`->schedule(r, s)`, where

(8.2.1) — `r` is a proxy for `rcvr` with base `system_context_replace-ability::receiver_proxy` (33.15 [exec.par.scheduler]) and

(8.2.2) — `s` is a preallocated backend storage for `r`.

```
template<class BulkSndr, class Env>     // exposition only
   see  below  bulk-transform(BulkSndr&&  bulk_sndr,  const  Env&
        env);
```

9. *Constraints*: `sender_in<BulkSndr, Env>` is `true` and either *`sender-for`*`<BulkSndr, bulk_chunked_t>` or *`sender-for`*`<BulkSndr, bulk_unchunked_t>` is `true`.

10. *Returns*: a prvalue `sndr` whose type models `sender` such that:

(10.1) — `get_completion_scheduler<set_value_t>(get_env(sndr))` is equal to `*this`.

(10.2) — `bulk_sndr` is connected to an unspecified receiver if a receiver `rcvr` is connected to `sndr`. If the resulting operation state is started,

(10.2.1) — If `bulk_sndr` completes with values `vals`, let `args` be a pack of lvalue subexpressions designating objects decay-copied from `vals`. Then

(10.2.1.1) — If `bulk_sndr` is the result of calling `bulk_chunked(child, policy, shape, f)`, *`sch`*`->schedule_bulk_chunked(shape, r, s)` is called where `r` is a bulk chunked proxy for `rcvr` with callable `f` and arguments `args`, and `s` is a preallocated backend storage for `r`.

(10.2.1.2) — Otherwise, `bulk_sndr` is the result of calling `bulk_unchunked(child, policy, shape, f)`. Calls *`sch`*`->schedule_bulk_unchunked(shape, r, s)` where `r` is a bulk unchunked proxy for `rcvr` with callable `f` and arguments `args`, and `s` is a preallocated backend storage for `r`.

(10.2.2) — All other completion operations are forwarded unchanged.

[Editor's note: In 33.15 [exec.par.scheduler], add a new paragraph after paragraph 3, another before paragraph 10, and change paragraphs 10 and 11 as follows:]

3. The expression `get_forward_progress_guarantee(sch)` returns `forward_progress_guarantee::parallel`.

?. The expression `get_completion_scheduler<set_value_t>(get_env(schedule(sch))) == sch` evaluates to `true`.

… as before …

?. Let `sch` be a subexpression of type `parallel_scheduler`. For subexpressions `sndr` and `env`, if `tag_of_t<Sndr>` is neither `bulk_chunked_t` nor `bulk_unchunked_t`, the expression `sch.`*`bulk-transform`*`(sndr, env)` is ill-formed; otherwise, let `child`, `pol`, `shape`, and `f` be subexpressions equal to the arguments used to create `sndr`.

10. ~~`parallel_scheduler` provides a customized implementation of the `bulk_chunked` algorithm (33.9.12.11 [exec.bulk]). If a receiver `rcvr` is connected to the sender returned by `bulk_chunked(sndr, pol,`~~

~~shape, f)~~ When the tag type of sndr is bulk_chunked_t, the expression sch.*bulk-transform*(sndr, env) returns a sender such that if it is connected to a receiver rcvr and the resulting operation state is started, then:

(10.1) — If ~~sndr~~child completes with values vals, let args be a pack of lvalue subexpressions designating vals, then b.schedule_bulk_chunked(shape, r, s) is called, where

(10.1.1) — r is a bulk chunked proxy for rcvr with callable f and arguments args and

(10.1.2) — s is a preallocated backend storage for r.

(10.2) — All other completion operations are forwarded unchanged.

[*Note:* Customizing the behavior of bulk_chunked affects the ~~default~~ implementation of bulk. — *end note*]

11. ~~parallel_scheduler provides a customized implementation of the bulk_unchunked algorithm (33.9.12.11 [exec.bulk]). If a receiver rcvr is connected to the sender returned by bulk_unchunked(sndr, pol, shape, f)~~ When the tag type of sndr is bulk_unchunked_t, the expression sch.*bulk-transform*(sndr, env) returns a sender such that if it is connected to a receiver rcvr and the resulting operation state is started, then:

(11.1) — If ~~sndr~~child completes with values vals, let args be a pack of lvalue subexpressions designating vals, then b.schedule_bulk_unchunked(shape, r, s) is called, where

(11.1.1) — r is a bulk unchunked proxy for rcvr with callable f and arguments args and

(11.1.2) — s is a preallocated backend storage for r.

(11.2) — All other completion operations are forwarded unchanged.

§ 6 Appendix A: Listing for updated transform_sender

The proposal requires some changes to how transform_sender operates. This new transform_sender still accepts a sender and an environment like the current one, but it no longer accepts a domain. It computes the two domains, starting and completing, and applies the two transforms, recursing if a transform changes the type of the sender.

The implementation of transform_sender might look something like this:

```
template<class A, class B>
concept same-decayed = std::same_as<std::decay_t<A>, std::de-
        cay_t<B>>;

template<class Domain, class Tag>
struct transform-sender-recurse
{
  template<class Sndr, class Env>
  using result-t =
      decltype(Domain().transform_sender(Tag(), declval<Sndr>(),
        declval<const Env&>()));

  constexpr transform-sender-recurse(Domain, Tag) noexcept {}

  template<class Sndr, class Env>
   decltype(auto) operator()(this auto self, Sndr&& sndr, const
        Env& env)
  {
    if constexpr (!requires { typename result-t<Sndr, Env>; })
    {
       // Domain does not have a transform_sender for this sndr
         so use default_domain instead.
          return transform-sender-recurse<default_domain, Tag>()
        (forward<Sndr>(sndr), env);
    }
    else if constexpr (same-decayed<Sndr, result-t<Sndr, Env>>)
    {
       // Domain can transform the sender but its type does not
         change. End recursion.
      return Domain().transform_sender(Tag(), std::forward<Sndr>
        (sndr), env);
    }
    else if constexpr (same_as<Tag, start_t>)
    {
      // The starting domain cannot change, so recurse on Domain
          return self(Domain().transform_sender(start, std::for-
        ward<Sndr>(sndr), env), env);
    }
    else
    {
       // The type of sndr changes after being transformed, so
         the type of the completion
      // domain could change too. Recurse on the (possibly) new
         domain:
      using attrs_t  = env_of_t<result-t<Sndr, Env>>;
       using domain_t = decltype(get_completion_domain<set_val-
        ue_t>(declval<attrs_t>(), env));

      return transform-sender-recurse<domain_t, Tag>()(
         Domain().transform_sender(set_value, std::forward<Sndr>
        (sndr), env), env);
    }
  }
};

template<class Sndr, class Env>
auto transform_sender(Sndr&& sndr, const Env& env)
```

```
{
  auto starting_domain    = get_domain(env);
   auto complete_domain     = get_completion_domain<set_value_t>
        (get_env(sndr), env);

   auto starting_transform = transform-sender-recurse(starting_-
        domain, start);
   auto complete_transform = transform-sender-recurse(complete_-
        domain, set_value);

        return    starting_transform(complete_transform(std::for-
        ward<Sndr>(sndr), env), env);
}
```

With this definition of `transform_sender`, `connect(sndr, rcvr)` is equivalent to `transform_sender(sndr, get_env(rcvr)).connect(rcvr)`, except that `rcvr` is evaluated only once.

§ 7 References

[P3325R1] Eric Niebler. 2024-07-14. A Utility for Creating Execution Environments.
https://wg21.link/p3325r1

[P3388R2] Robert Leahy. 2025-04-01. When Do You Know connect Doesn't Throw?
https://wg21.link/p3388r2

[P3718R0] Eric Niebler. 2025-06-28. Fixing Lazy Sender Algorithm Customization, Again.
https://wg21.link/p3718r0